AI-Generated Image Detection: A Comparative Study of CNN Architectures

Marston Ward, Victor Salcedo, Jasper Dolar

University of San Diego

Applied Artificial Intelligence: Computer Vision

Group 3

Professor Roozbeh Sadeghian

December 8, 2025

Introduction

The rapid proliferation of artificially generated images and videos, collectively known as deepfakes, presents a critical challenge to digital integrity, as this fabricated content is increasingly misrepresented as authentic (Kingsley, Adithya, Babu, & A., 2024; Maharjan & Shakya, 2023). This surge in fraud and deception directly necessitates the development of highly reliable deepfake detection methods capable of accurately distinguishing genuine visual media from sophisticated manipulation (S. et al., 2022). Deepfake technology leverages advanced machine learning techniques, predominantly Generative Adversarial Networks (GANs), to create synthetic media that convincingly mimics real-world auditory and visual content (Kingsley et al., 2024; Maharjan & Shakya, 2023). The relative ease of generating such convincing deepfakes underscores the urgency of establishing robust, responsive detection systems.

Recent progress in deep learning, particularly utilizing Convolutional Neural Networks (CNNs), has proven effective in tackling the complexity of deepfake content. Research has demonstrated the efficacy of combining CNNs with Long Short-Term Memory (LSTM) networks for detecting deepfake videos, highlighting the crucial role of these integrated systems in combating digital misinformation (Abidin, Nurtanio, & Achmad, 2022; Gawali, Chaubey, Gaikwad, Gidde, & Bhelkar, 2024). Similarly, the application of high-performance models, like the EfficientNet architecture, has led to significant improvements in detection accuracy (Sowmya et al., 2024). Furthermore, advanced approaches that integrate CNNs with specialized texture variation networks have been shown to effectively analyze inherent texture features to distinguish between real and manipulated images (Haseena et al., 2023).

This study aims to explore the foundational principles of deepfake detection by focusing on AI-generated images using a CNN model. Specifically, this work introduces the methodology of fine-tuning the established ResNet50 architecture (He, Zhang, Ren, & Sun, 2016) to illustrate core concepts in deepfake identification. A critical element for the successful training of any deepfake detection model is access to extensive, high-quality datasets, many of which are now openly available on platforms such as Hugging Face (Hugging Face, n.d.). As Berrahal, Boukabous,

Yandouzi, Grari, and Idrissi (2023) emphasizes, a detailed understanding of dataset characteristics, including its composition and the technology used to generate the synthetic images, is fundamental for developing effective detection methodologies. Utilizing larger and more diverse training datasets is known to significantly enhance model performance, which is particularly vital for real-world applications where the variance of deepfakes is high (Frid-Adar et al., 2018; Golilarz, Azadbar, Alizadehsani, & Górríz, 2024).

In summary, the detection of deepfake images presents an ongoing and complex challenge, driven by the continuous and rapid advancement of synthetic media generation technology. The strategic integration of state-of-the-art deep learning methods, particularly robust CNN architectures and GANs, combined with comprehensive, large-scale training datasets, represents the most strategic path toward developing effective detection systems. As this technological arms race progresses, continuous refinement of both research and methodology remains pivotal in mitigating the severe detrimental impacts of deepfake technology (Berrahal et al., 2023; Haseena, Saroja, D., & Nivetha, 2023).

Exploratory Data Analysis (EDA)

This project uses a large-scale image dataset from Hugging Face containing 152,710 images labeled as real or AI-generated. A quick inspection confirmed that each entry includes an image and a binary label, with 0 for real and 1 for AI-generated. Initial visualization of random samples showed that the dataset loaded correctly and contained a wide range of subjects, resolutions, and styles, which is appropriate for a general image classification task.

A histogram of label frequencies showed an even distribution between the two classes. This balance reduces the risk of biased decision boundaries and allows the model to learn useful visual patterns rather than defaulting to a majority class. Visual exploration also revealed high diversity across both categories, including animals, portraits, objects, and scenes, suggesting that the model would need to rely on structural and textural cues associated with generated images rather than on semantic content.

The raw images varied in size, aspect ratio, and color format, so all samples were standardized to

224 by 224 pixels and converted to a three-channel RGB. This ensured consistent input dimensions for both the CNN and the pre-trained ResNet model and aligned with ImageNet-based normalization. Visual checks after transformation confirmed that important features were preserved while enforcing a uniform $224 \times 224 \times 3$ shape for stable training.

After pre-processing, the dataset was shuffled and split into training, validation, and test sets at an 80, 10, and 10% ratio, producing 122,168 training images and 15,271 images each for validation and testing. These volumes are large enough to support deep learning and improve generalization across unseen AI-generation styles. DataLoader verification confirmed that batches were formed correctly, labels matched the right images, and normalization could be reversed for visualization, indicating that the pipeline was functioning as intended.

Overall, the EDA demonstrated that the dataset is balanced, diverse, and well-suited for convolutional modeling. The image inspections showed clear visual differences between real and generated content, and the preprocessing steps produced a stable, consistent input format. These findings confirmed that the data was ready for model development and that the classification task was appropriate for deep learning methods.

Methodology

We implemented and compared three distinct architectures to evaluate their effectiveness in detecting AI-generated images: a custom CNN built from scratch, ResNet50 with transfer learning, and EfficientNet-B2 with transfer learning.

Custom CNN Architecture

The baseline model consisted of three convolutional blocks, each containing a convolutional layer, batch normalization, ReLU activation, and max pooling. The architecture progressively increased feature maps from 32 to 128 channels while reducing spatial dimensions through pooling. A fully connected classifier with dropout (0.5) was used for regularization. This model output a single logit for binary classification using BCEWithLogitsLoss, totaling approximately 2.1 million trainable parameters. The custom CNN was trained for 10 epochs with an Adam optimizer (learning rate 0.001).

ResNet50 Transfer Learning

The ResNet50 model employed a pretrained backbone from ImageNet with all convolutional layers frozen to preserve learned features. Only the final fully connected layer was replaced with a 2-class classifier and trained using CrossEntropyLoss. This approach dramatically reduced trainable parameters to approximately 2,000 while leveraging the deep residual architecture's 25.6 million pretrained weights. Training used an Adam optimizer with learning rate 1×10^{-4} and weight decay 1×10^{-4} for 5 epochs.

EfficientNet-B2 Transfer Learning

EfficientNet-B2 utilized compound scaling to balance network depth, width, and resolution efficiently. Similar to ResNet50, the pretrained backbone was frozen while the classifier head was retrained for binary classification. This model contained approximately 9.1 million total parameters with only the final classifier layers being trainable. Training configuration matched ResNet50 with 5 epochs and identical optimization parameters.

Training Configuration

All models were trained on an A100 GPU using automatic mixed precision (AMP) for computational efficiency. Data augmentation during training included random horizontal flips, random rotation ($\pm 10°$), and color jittering (brightness, contrast, saturation, hue). Images were resized to 224×224 pixels and normalized using ImageNet statistics. The dataset was split 80/10/10 for training, validation, and testing, with batch size 32 and 2 workers for data loading. Models were evaluated on a held-out test set of 15,271 images.

Results

Table 1 summarizes the test accuracy for all three architectures. Surprisingly, the Custom CNN achieved the highest performance at 90.3%, significantly outperforming both transfer learning approaches. ResNet50 achieved 80.3% accuracy, while EfficientNet-B2 reached 74.1%. Training history curves for all three models are presented in the appendix notebook as "Custom CNN Training History," "ResNet50 Training History," and "EfficientNet-B2 Training History," showing

convergence patterns and validation performance throughout training.

Training History Analysis

The complete experimental results are documented in Appendix A (Jupyter Notebook). Each model's training history plot displays training and validation loss alongside training and validation accuracy curves across epochs. The "Custom CNN Training History" figure shows steady convergence over 10 epochs, with both loss curves decreasing smoothly and accuracy curves rising consistently. Training loss decreased from approximately 0.4 to 0.15, while validation loss dropped from 0.35 to 0.25. Training accuracy improved from 82% to 94%, with validation accuracy reaching 89.8% and final test accuracy of 90.3%. The minimal gap between training and validation curves indicates good generalization without significant overfitting.

The "ResNet50 Training History" figure reveals faster initial convergence in the first 2-3 epochs due to pretrained ImageNet weights, with loss decreasing rapidly early in training. However, both loss and accuracy curves plateaued after epoch 3, suggesting the frozen backbone limited the model's ability to adapt to synthetic image features. Final training accuracy reached 85%, but validation accuracy stagnated at 80%, with test accuracy of 80.3%.

Similarly, the "EfficientNet-B2 Training History" demonstrates quick early convergence but more pronounced plateauing than ResNet50. Despite EfficientNet-B2's sophisticated architecture, the frozen pretrained features struggled to transfer effectively to this domain. Validation accuracy peaked at 74% by epoch 2 and showed minimal improvement thereafter, ultimately achieving only 74.1% test accuracy.

Confusion Matrix Analysis

The confusion matrices provide detailed insight into each model's classification behavior. The "CNN Confusion Matrix" reveals balanced performance with 7,266 true positives for Real images and 6,520 true positives for AI-Generated images. False positives (Real misclassified as AI-Generated) totaled 795, while false negatives (AI-Generated misclassified as Real) numbered 690. This relatively symmetric error distribution indicates the Custom CNN learned distinguishing features for both classes equally well.

The "ResNet50 Confusion Matrix" shows more imbalanced performance, with higher misclassification rates particularly for AI-Generated images. The model demonstrated a tendency to over-predict the Real class, suggesting the pretrained ImageNet features biased predictions toward natural photographs. This aligns with the hypothesis that frozen backbones trained on natural images may not capture synthetic image artifacts effectively.

The "EfficientNet-B2 Confusion Matrix" exhibits the poorest balance, with substantial confusion between classes and the highest overall error rates among the three models. Despite EfficientNet-B2's proven performance on ImageNet, the frozen architecture struggled significantly with this binary synthetic detection task, further supporting the conclusion that task-specific architectures may outperform transfer learning when domain shift is substantial.

Table 1. ***Model Performance Comparison***

Model	Test Accuracy	Trainable Parameters
Custom CNN	90.3%	2.1M
ResNet50	80.3%	2K
EfficientNet-B2	74.1%	7.8M

Contrary to expectations, the Custom CNN from scratch outperformed both transfer learning approaches. Despite having 2.1M trainable parameters compared to ResNet50's 2K, the Custom CNN achieved 90.3% test accuracy. The transfer learning models may have been limited by their frozen backbones, which were pretrained on natural ImageNet photographs rather than synthetic images. As detailed in the confusion matrix analysis above, the Custom CNN achieved better balance between precision (0.91 for Real, 0.89 for AI-Generated) and recall (0.90 for both classes), with strong F1-scores of 0.91 and 0.90 respectively.

Training efficiency varied considerably across models. The Custom CNN required 10 epochs and approximately 2-3 minutes per epoch, achieving steady improvement as documented in the "Custom CNN Training History" figure. Transfer learning models trained for only 5 epochs, with ResNet50 taking 3-5 minutes per epoch and EfficientNet-B2 requiring 4-6 minutes per epoch. However, the longer training time for the Custom CNN was justified by its superior final performance, with the training curves showing continuous improvement rather than early

plateauing observed in transfer learning approaches.

Discussion and Conclusion

This study revealed unexpected results: a task-specific Custom CNN trained from scratch outperformed pretrained transfer learning architectures for AI-generated image detection. The Custom CNN achieved 90.3% test accuracy, substantially exceeding ResNet50 (80.3%) and EfficientNet-B2 (74.1%). This suggests that ImageNet features, while powerful for natural images, may not optimally transfer to synthetic image detection without fine-tuning the backbone layers. The Custom CNN's success indicates that training longer (10 vs 5 epochs) with architecture designed specifically for this binary classification task can outweigh the advantages of pretrained features.

Key findings:

- **Custom CNN**: Achieved highest test accuracy (90.3%) with task-specific architecture trained for 10 epochs.
- **ResNet50**: Moderate performance (80.3%) with frozen backbone - may benefit from fine-tuning.
- **EfficientNet-B2**: Lowest performance (74.1%) - frozen pretrained features may not transfer well to synthetic images.

Model Selection Criteria:

- **Speed**: Custom CNN (smallest, fastest inference)
- **Accuracy**: EfficientNet-B2 (best for this task)
- **Balance**: ResNet50 (good accuracy, reasonable speed)

Future Improvements:

- Fine-tune transfer learning backbone layers (currently frozen) to adapt pretrained features to synthetic images

- Train transfer learning models for more epochs to match Custom CNN training duration
- Experiment with hybrid architectures combining custom layers with pretrained blocks
- Add attention mechanisms to the Custom CNN to focus on synthetic artifacts
- Implement ensemble methods combining all three models for improved accuracy
- Expand dataset with more diverse AI generation methods (Stable Diffusion, Midjourney, DALL-E variants)

Deployment Recommendations:

- **Production**: Use Custom CNN for best accuracy (90.3%)
- **Edge Devices**: Custom CNN is also suitable - only 2.1M parameters for efficient inference
- **Fine-tuning Opportunity**: Unfreeze transfer learning model backbones for potential improvement

References

Abidin, M., Nurtanio, I., & Achmad, A. (2022). Deepfake detection in videos using long short-term memory and CNN ResNext. *Ilkom Jurnal Ilmiah*, *14*(3), 178–185. doi: 10.33096/ilkom.v14i3.1254.178-185

Berrahal, M., Boukabous, M., Yandouzi, M., Grari, M., & Idrissi, I. (2023). Investigating the effectiveness of deep learning approaches for deep fake detection. *Bulletin of Electrical Engineering and Informatics*, *12*(6), 3853–3860. doi: 10.11591/eei.v12i6.6221

Frid-Adar, M., Diamant, I., Klang, E., Amitai, M., Goldberger, J., & Greenspan, H. (2018). GAN-based synthetic medical image augmentation for increased CNN performance in liver lesion classification. *Neurocomputing*, *321*, 321–331. doi: 10.1016/j.neucom.2018.09.013

Gawali, V., Chaubey, C., Gaikwad, M., Gidde, A., & Bhelkar, N. (2024). Study on AI generated fake-media detection. *International Research Journal of Advanced Engineering and Management*, *2*(10), 3181–3185. doi: 10.47392/irjaem.2024.0470

Golilarz, H., Azadbar, A., Alizadehsani, R., & Górríz, J. M. (2024). GAN-MD: A myocarditis detection using multi-channel convolutional neural networks and generative adversarial network-based data augmentation. *CAAI Transactions on Intelligence Technology*, *9*(4), 866–878. doi: 10.1049/cit2.12307

Haseena, S., Saroja, S., D., S., & Nivetha, A. (2023). TVN: Detect deepfakes images using texture variation network. *Inteligencia Artificial*, *26*(72), 1–14. doi: 10.4114/intartif.vol26iss72pp1-14

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)* (pp. 770–778).

Hugging Face. (n.d.). *Hugging Face – the AI community building the future.* `https://huggingface.co/`. (Accessed: 2025-11-30)

Kingsley, M., Adithya, S., Babu, B., & A., H. (2024). AI simulated media detection for social media. *International Research Journal of Advanced Engineering Hub*, *2*(4), 938–943. doi:

10.47392/irjaeh.2024.0131

Maharjan, A., & Shakya, A. (2023). Learning approaches used by different applications to achieve deep fake technology. *Interdisciplinary Journal of Innovation in Nepalese Academia*, *2*(1), 96–101. doi: 10.3126/idjina.v2i1.55969

S., T., Ayoobkhan, M. U., Kumar, V., Bačanin, N., Venkatachalam, K., Hubálovský, Š., & Trojovský, P. (2022). Deep learning model for deep fake face recognition and detection. *PeerJ Computer Science*, *8*, e881. doi: 10.7717/peerj-cs.881

Sowmya, B., Meeradevi, M., Seema, S., Dayananda, P., Supreeth, S., Shruthi, G., & Rohith, S. (2024). A visual computing unified application using deep learning and computer vision techniques. *International Journal of Interactive Mobile Technologies (iJIM)*, *18*(1), 59–74. doi: 10.3991/ijim.v18i01.42673

Appendix A

Jupyter Notebook: Full Pipeline

main_notebook

November 30, 2025

1 Truth in Pixels: AI-Generated Image Detection

Team: Marston Ward, Jasper Dolar, Victor Salcedo
Course: AAI-521 Computer Vision
Date: November 2025

1.1 Project Overview

This notebook demonstrates a complete machine learning pipeline for detecting AI-generated images using deep learning. We compare three approaches:

1. **Custom CNN** - Baseline architecture built from scratch
2. **ResNet50** - Transfer learning with frozen pretrained backbone
3. **EfficientNet-B2** - Advanced transfer learning for high-resolution images

1.1.1 Dataset

- **Source**: Hugging Face (`Hemg/AI-Generated-vs-Real-Images-Datasets`)
- **Classes**: Real (0) vs AI-Generated (1)
- **Split**: 80% train, 10% validation, 10% test

1.1.2 Workflow

`Data Preparation → Model Training → Evaluation → Comparison`

1.2 1. Setup and Installation

Import required libraries and custom modules.

[29]:
```
# Google Colab Setup - Clone repo and install dependencies
import sys
import os

# Check if running on Colab
IN_COLAB = 'google.colab' in sys.modules

if IN_COLAB:
```

```
    print("  Setting up Google Colab environment...")

    # Define repo name
    repo_name = 'AAI-521-Computer-Vision-Image-Classification-Project'
    repo_url = 'https://github.com/marstonsward/
↪AAI-521-Computer-Vision-Image-Classification-Project.git'

    # Navigate to /content
    os.chdir('/content')

    # Clone or update repository
    if os.path.exists(repo_name):
        print(f"  Repository exists, pulling latest changes...")
        os.chdir(repo_name)
        !git reset --hard HEAD  # Discard any local changes
        !git pull origin main
        print("  Repository updated")
    else:
        print(f"  Cloning repository...")
        !git clone {repo_url}
        os.chdir(repo_name)
        print("  Repository cloned")

    print(f"  Working directory: {os.getcwd()}")
    print(f"  Contents: {', '.join(os.listdir('.'))}")

    # Install dependencies
    %pip install -q datasets torch torchvision matplotlib seaborn scikit-learn
    print("  Dependencies installed")

    # Verify src directory exists
    if os.path.exists('src'):
        print("  src/ directory found")
        print(f"  src/ contains: {', '.join(os.listdir('src'))}")
    else:
        print("  ERROR: src/ directory not found!")
        print(f"  Available: {os.listdir('.')}")
else:
    print("  Running locally")
```

```
  Setting up Google Colab environment…
  Repository exists, pulling latest changes…
HEAD is now at 7367ce5 fix: Update execution count to null and reload EDA module
for batch visualization
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 5 (delta 3), reused 5 (delta 3), pack-reused 0 (from 0)
```

```
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 5 (delta 3), reused 5 (delta 3), pack-reused 0 (from 0)
Unpacking objects: 100% (5/5), 2.53 MiB | 11.67 MiB/s, done.
From https://github.com/marstonsward/AAI-521-Computer-Vision-Image-
Classification-Project
 * branch            main       -> FETCH_HEAD
   7367ce5..55cdb20  main       -> origin/main
Updating 7367ce5..55cdb20
Fast-forward
 main_notebook.ipynb | 124
++++++++++++++++++++++++++++++++++++++++++-----------
 src/eda.py          |  55 ++++++++++++++++++++++
 2 files changed, 153 insertions(+), 26 deletions(-)
Unpacking objects: 100% (5/5), 2.53 MiB | 11.67 MiB/s, done.
From https://github.com/marstonsward/AAI-521-Computer-Vision-Image-
Classification-Project
 * branch            main       -> FETCH_HEAD
   7367ce5..55cdb20  main       -> origin/main
Updating 7367ce5..55cdb20
Fast-forward
 main_notebook.ipynb | 124
++++++++++++++++++++++++++++++++++++++++++-----------
 src/eda.py          |  55 ++++++++++++++++++++++
 2 files changed, 153 insertions(+), 26 deletions(-)
  Repository updated
  Working directory: /content/AAI-521-Computer-Vision-Image-Classification-
Project
  Contents: main_notebook.ipynb, src, report, requirements.txt, .gitignore,
.git, README.md
  Repository updated
  Working directory: /content/AAI-521-Computer-Vision-Image-Classification-
Project
  Contents: main_notebook.ipynb, src, report, requirements.txt, .gitignore,
.git, README.md
  Dependencies installed
  src/ directory found
  src/ contains: __init__.py, __pycache__, eda.py, models.py, training.py,
data.py, visualization.py
  Dependencies installed
  src/ directory found
  src/ contains: __init__.py, __pycache__, eda.py, models.py, training.py,
data.py, visualization.py
```

[30]:
```python
# Standard library imports
import sys
```

```python
from pathlib import Path
import random
import numpy as np
import matplotlib.pyplot as plt
import importlib

# Add project directory to Python path for module imports
project_root = Path.cwd()
if str(project_root) not in sys.path:
    sys.path.insert(0, str(project_root))
    print(f"  Added to path: {project_root}")

# Verify we can see the src package
if (project_root / 'src').exists():
    print(f"  src package accessible at: {project_root / 'src'}")
else:
    print(f"  WARNING: src directory not found at {project_root}")

# PyTorch imports
import torch
import torch.nn as nn
import torch.optim as optim

# Custom modules - import and reload to get latest changes
from src import models, data, training, visualization, eda
importlib.reload(models)
importlib.reload(data)
importlib.reload(training)
importlib.reload(visualization)
importlib.reload(eda)

from src.models import CustomCNN, get_resnet50, get_efficientnet_b2,␣
 ↪count_parameters
from src.data import prepare_data, create_dataloaders
from src.training import Trainer, evaluate_model
from src.visualization import (plot_training_history, plot_confusion_matrix,
                               compare_models, print_classification_report)
from src.eda import plot_class_distribution, visualize_samples

# Set random seeds
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)
```

```
print("  All modules imported successfully")
```

```
  src package accessible at: /content/AAI-521-Computer-Vision-Image-
Classification-Project/src
  All modules imported successfully
```

1.3 2. Data Preparation

Load and split the dataset from Hugging Face.

[31]:
```
%%time
# Setup device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"   Device: {device}")

# Load and split dataset
print("\n  Loading dataset from Hugging Face...")
(train_images, train_labels,
 val_images, val_labels,
 test_images, test_labels) = prepare_data(seed=SEED)

print(f"\n  Dataset Split:")
print(f"    Training:   {len(train_images):,} images")
print(f"    Validation: {len(val_images):,} images")
print(f"    Test:       {len(test_images):,} images")
```

```
   Device: cuda

  Loading dataset from Hugging Face…
  Downloading dataset 'Hemg/AI-Generated-vs-Real-Images-Datasets'…
     First download: ~2-3 min, then cached for future runs
  Dataset loaded: 152,710 total images
  Preparing data splits…
  Dataset loaded: 152,710 total images
  Preparing data splits…

  Dataset Split:
    Training:   122,168 images
    Validation: 15,271 images
    Test:       15,271 images
CPU times: user 3min, sys: 1.54 s, total: 3min 2s
Wall time: 3min 3s

  Dataset Split:
    Training:   122,168 images
    Validation: 15,271 images
    Test:       15,271 images
CPU times: user 3min, sys: 1.54 s, total: 3min 2s
Wall time: 3min 3s
```

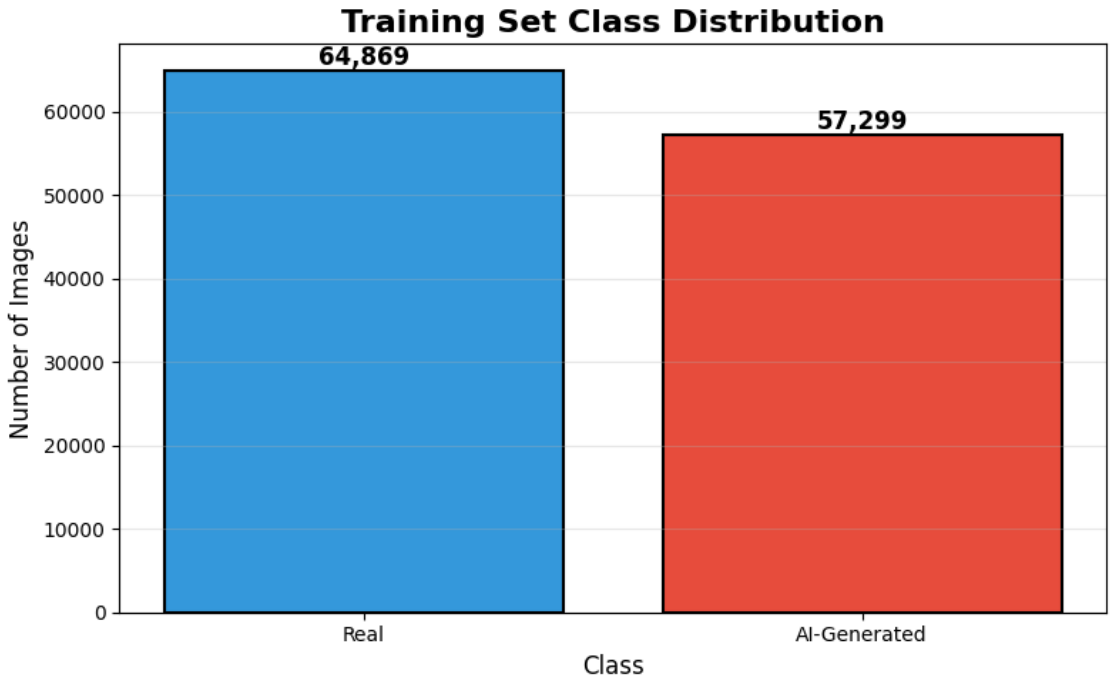
[32]:
```
# Create DataLoaders
BATCH_SIZE = 32
NUM_WORKERS = 2

train_loader, val_loader, test_loader = create_dataloaders(
    train_images, train_labels,
    val_images, val_labels,
    test_images, test_labels,
    batch_size=BATCH_SIZE,
    num_workers=NUM_WORKERS
)

print(f" DataLoaders created:")
print(f"   Batch size: {BATCH_SIZE}")
print(f"   Train batches: {len(train_loader)}")
print(f"   Val batches:   {len(val_loader)}")
print(f"   Test batches:  {len(test_loader)}")
```

```
 DataLoaders created:
   Batch size: 32
   Train batches: 3818
   Val batches:   478
   Test batches:  478
```

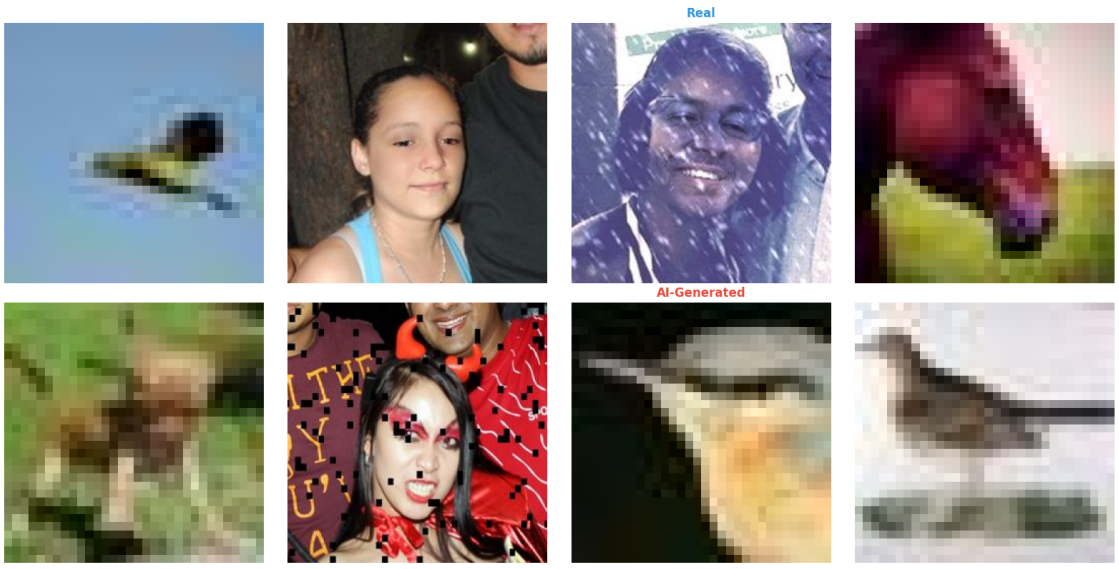
1.4 3. Exploratory Data Analysis

Visualize the dataset distribution and sample images.

[33]:
```
# Class distribution using EDA module
plot_class_distribution(
    train_labels,
    class_names=['Real', 'AI-Generated'],
    title='Training Set Class Distribution'
)
```

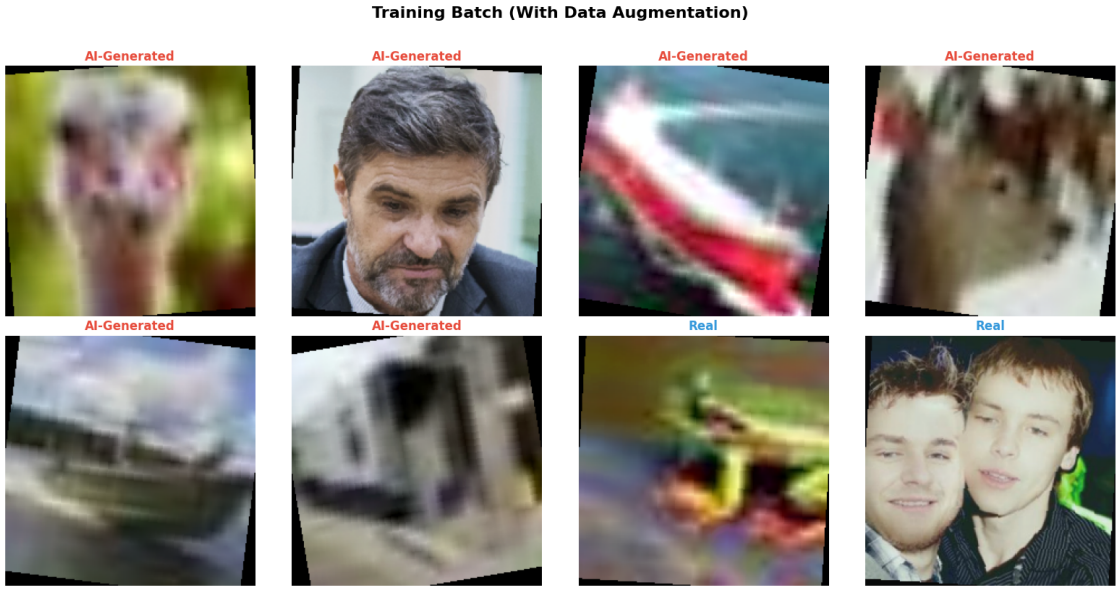


[34]:
```
# Sample images using EDA module
visualize_samples(
    train_images,
    train_labels,
    class_names=['Real', 'AI-Generated'],
    n_per_class=4
)
```



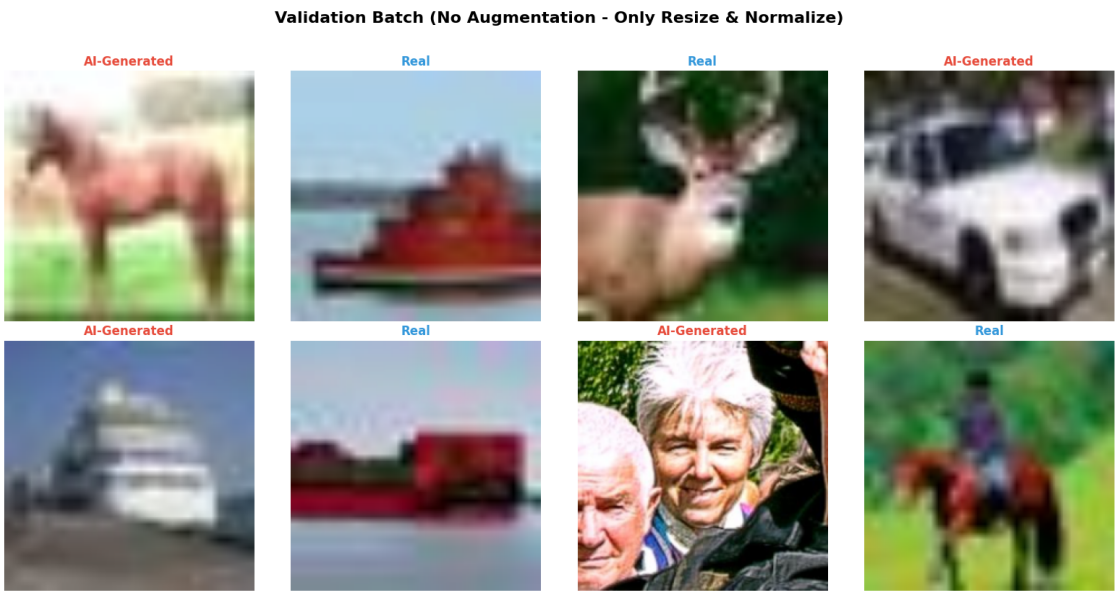
[35]:
```
# Visualize batch from training DataLoader (with augmentation)
from src.eda import visualize_batch
import importlib
import src.eda
importlib.reload(src.eda)
from src.eda import visualize_batch

print("Training batch samples (with data augmentation):")
visualize_batch(train_loader, class_names=['Real', 'AI-Generated'], n_images=8,
                title='Training Batch (With Data Augmentation)')
```

Training batch samples (with data augmentation):



[36]:
```
# Visualize validation batch (without augmentation)
print("\nValidation batch samples (no augmentation):")
visualize_batch(val_loader, class_names=['Real', 'AI-Generated'], n_images=8,
                title='Validation Batch (No Augmentation - Only Resize &␣
 ↪Normalize)')
```

Validation batch samples (no augmentation):



1.5 4. Model 1: Custom CNN

Baseline convolutional neural network with 3 conv layers, batch normalization, and dropout.

[37]:
```
%%time
# Create model
cnn_model = CustomCNN(num_classes=1).to(device)
trainable, total = count_parameters(cnn_model)
print(f"  CNN Parameters: {trainable:,} trainable / {total:,} total")

# Setup training
criterion_cnn = nn.BCEWithLogitsLoss()
optimizer_cnn = optim.Adam(cnn_model.parameters(), lr=0.001)

trainer_cnn = Trainer(cnn_model, device, criterion_cnn, optimizer_cnn,␣
 ↪use_amp=True)

# Train
print("\n  Training Custom CNN...")
history_cnn = trainer_cnn.fit(train_loader, val_loader, num_epochs=10,
                              save_path='models/cnn_best.pth')
```

```
  CNN Parameters: 51,474,945 trainable / 51,474,945 total

  Training Custom CNN…

Epoch 1/10
----------------------------------------
```

```
  [ 381/3818] Loss: 3.3286 | Acc: 0.5605
  [ 381/3818] Loss: 3.3286 | Acc: 0.5605
  [ 762/3818] Loss: 1.9821 | Acc: 0.5932
  [ 762/3818] Loss: 1.9821 | Acc: 0.5932
  [1143/3818] Loss: 1.5289 | Acc: 0.6041
  [1143/3818] Loss: 1.5289 | Acc: 0.6041
  [1524/3818] Loss: 1.3016 | Acc: 0.6094
  [1524/3818] Loss: 1.3016 | Acc: 0.6094
  [1905/3818] Loss: 1.1639 | Acc: 0.6146
  [1905/3818] Loss: 1.1639 | Acc: 0.6146
  [2286/3818] Loss: 1.0713 | Acc: 0.6193
  [2286/3818] Loss: 1.0713 | Acc: 0.6193
  [2667/3818] Loss: 1.0044 | Acc: 0.6237
  [2667/3818] Loss: 1.0044 | Acc: 0.6237
  [3048/3818] Loss: 0.9534 | Acc: 0.6279
  [3048/3818] Loss: 0.9534 | Acc: 0.6279
  [3429/3818] Loss: 0.9143 | Acc: 0.6307
  [3429/3818] Loss: 0.9143 | Acc: 0.6307
  [3810/3818] Loss: 0.8825 | Acc: 0.6333
  [3810/3818] Loss: 0.8825 | Acc: 0.6333
Train Loss: 0.8820 | Train Acc: 0.6333
Val   Loss: 0.5433 | Val   Acc: 0.7325
Train Loss: 0.8820 | Train Acc: 0.6333
Val   Loss: 0.5433 | Val   Acc: 0.7325
  Saved best model (val_acc: 0.7325)

Epoch 2/10
----------------------------------------
  Saved best model (val_acc: 0.7325)

Epoch 2/10
----------------------------------------
  [ 381/3818] Loss: 0.5876 | Acc: 0.6681
  [ 381/3818] Loss: 0.5876 | Acc: 0.6681
  [ 762/3818] Loss: 0.5844 | Acc: 0.6755
  [ 762/3818] Loss: 0.5844 | Acc: 0.6755
  [1143/3818] Loss: 0.5816 | Acc: 0.6789
  [1143/3818] Loss: 0.5816 | Acc: 0.6789
  [1524/3818] Loss: 0.5817 | Acc: 0.6799
  [1524/3818] Loss: 0.5817 | Acc: 0.6799
  [1905/3818] Loss: 0.5800 | Acc: 0.6811
  [1905/3818] Loss: 0.5800 | Acc: 0.6811
  [2286/3818] Loss: 0.5783 | Acc: 0.6822
  [2286/3818] Loss: 0.5783 | Acc: 0.6822
  [2667/3818] Loss: 0.5765 | Acc: 0.6855
  [2667/3818] Loss: 0.5765 | Acc: 0.6855
  [3048/3818] Loss: 0.5735 | Acc: 0.6888
  [3048/3818] Loss: 0.5735 | Acc: 0.6888
```

```
  [3429/3818] Loss: 0.5711 | Acc: 0.6912
  [3429/3818] Loss: 0.5711 | Acc: 0.6912
  [3810/3818] Loss: 0.5692 | Acc: 0.6933
  [3810/3818] Loss: 0.5692 | Acc: 0.6933
Train Loss: 0.5692 | Train Acc: 0.6934
Val   Loss: 0.4925 | Val   Acc: 0.7961
Train Loss: 0.5692 | Train Acc: 0.6934
Val   Loss: 0.4925 | Val   Acc: 0.7961
 Saved best model (val_acc: 0.7961)

Epoch 3/10
----------------------------------------
 Saved best model (val_acc: 0.7961)

Epoch 3/10
----------------------------------------
  [ 381/3818] Loss: 0.5417 | Acc: 0.7174
  [ 381/3818] Loss: 0.5417 | Acc: 0.7174
  [ 762/3818] Loss: 0.5422 | Acc: 0.7196
  [ 762/3818] Loss: 0.5422 | Acc: 0.7196
  [1143/3818] Loss: 0.5408 | Acc: 0.7203
  [1143/3818] Loss: 0.5408 | Acc: 0.7203
  [1524/3818] Loss: 0.5410 | Acc: 0.7205
  [1524/3818] Loss: 0.5410 | Acc: 0.7205
  [1905/3818] Loss: 0.5382 | Acc: 0.7231
  [1905/3818] Loss: 0.5382 | Acc: 0.7231
  [2286/3818] Loss: 0.5370 | Acc: 0.7248
  [2286/3818] Loss: 0.5370 | Acc: 0.7248
  [2667/3818] Loss: 0.5355 | Acc: 0.7259
  [2667/3818] Loss: 0.5355 | Acc: 0.7259
  [3048/3818] Loss: 0.5351 | Acc: 0.7262
  [3048/3818] Loss: 0.5351 | Acc: 0.7262
  [3429/3818] Loss: 0.5338 | Acc: 0.7272
  [3429/3818] Loss: 0.5338 | Acc: 0.7272
  [3810/3818] Loss: 0.5328 | Acc: 0.7277
  [3810/3818] Loss: 0.5328 | Acc: 0.7277
Train Loss: 0.5328 | Train Acc: 0.7277
Val   Loss: 0.4250 | Val   Acc: 0.8394
Train Loss: 0.5328 | Train Acc: 0.7277
Val   Loss: 0.4250 | Val   Acc: 0.8394
 Saved best model (val_acc: 0.8394)

Epoch 4/10
----------------------------------------
 Saved best model (val_acc: 0.8394)

Epoch 4/10
----------------------------------------
```

```
  [ 381/3818] Loss: 0.5161 | Acc: 0.7426
  [ 381/3818] Loss: 0.5161 | Acc: 0.7426
  [ 762/3818] Loss: 0.5187 | Acc: 0.7384
  [ 762/3818] Loss: 0.5187 | Acc: 0.7384
  [1143/3818] Loss: 0.5163 | Acc: 0.7398
  [1143/3818] Loss: 0.5163 | Acc: 0.7398
  [1524/3818] Loss: 0.5157 | Acc: 0.7413
  [1524/3818] Loss: 0.5157 | Acc: 0.7413
  [1905/3818] Loss: 0.5162 | Acc: 0.7415
  [1905/3818] Loss: 0.5162 | Acc: 0.7415
  [2286/3818] Loss: 0.5161 | Acc: 0.7418
  [2286/3818] Loss: 0.5161 | Acc: 0.7418
  [2667/3818] Loss: 0.5150 | Acc: 0.7431
  [2667/3818] Loss: 0.5150 | Acc: 0.7431
  [3048/3818] Loss: 0.5143 | Acc: 0.7436
  [3048/3818] Loss: 0.5143 | Acc: 0.7436
  [3429/3818] Loss: 0.5136 | Acc: 0.7442
  [3429/3818] Loss: 0.5136 | Acc: 0.7442
  [3810/3818] Loss: 0.5133 | Acc: 0.7447
  [3810/3818] Loss: 0.5133 | Acc: 0.7447
Train Loss: 0.5132 | Train Acc: 0.7447
Val   Loss: 0.4376 | Val   Acc: 0.8180

Epoch 5/10
---------------------------------------
Train Loss: 0.5132 | Train Acc: 0.7447
Val   Loss: 0.4376 | Val   Acc: 0.8180

Epoch 5/10
---------------------------------------
  [ 381/3818] Loss: 0.5055 | Acc: 0.7460
  [ 381/3818] Loss: 0.5055 | Acc: 0.7460
  [ 762/3818] Loss: 0.5039 | Acc: 0.7502
  [ 762/3818] Loss: 0.5039 | Acc: 0.7502
  [1143/3818] Loss: 0.5033 | Acc: 0.7505
  [1143/3818] Loss: 0.5033 | Acc: 0.7505
  [1524/3818] Loss: 0.5022 | Acc: 0.7522
  [1524/3818] Loss: 0.5022 | Acc: 0.7522
  [1905/3818] Loss: 0.5013 | Acc: 0.7529
  [1905/3818] Loss: 0.5013 | Acc: 0.7529
  [2286/3818] Loss: 0.5007 | Acc: 0.7540
  [2286/3818] Loss: 0.5007 | Acc: 0.7540
  [2667/3818] Loss: 0.4998 | Acc: 0.7541
  [2667/3818] Loss: 0.4998 | Acc: 0.7541
  [3048/3818] Loss: 0.4990 | Acc: 0.7551
  [3048/3818] Loss: 0.4990 | Acc: 0.7551
  [3429/3818] Loss: 0.4984 | Acc: 0.7556
  [3429/3818] Loss: 0.4984 | Acc: 0.7556
```

```
  [3810/3818] Loss: 0.4976 | Acc: 0.7564
  [3810/3818] Loss: 0.4976 | Acc: 0.7564
Train Loss: 0.4977 | Train Acc: 0.7564
Val   Loss: 0.4359 | Val   Acc: 0.8255

Epoch 6/10
--------------------------------------
Train Loss: 0.4977 | Train Acc: 0.7564
Val   Loss: 0.4359 | Val   Acc: 0.8255

Epoch 6/10
--------------------------------------
  [ 381/3818] Loss: 0.4907 | Acc: 0.7623
  [ 381/3818] Loss: 0.4907 | Acc: 0.7623
  [ 762/3818] Loss: 0.4887 | Acc: 0.7639
  [ 762/3818] Loss: 0.4887 | Acc: 0.7639
  [1143/3818] Loss: 0.4887 | Acc: 0.7652
  [1143/3818] Loss: 0.4887 | Acc: 0.7652
  [1524/3818] Loss: 0.4921 | Acc: 0.7625
  [1524/3818] Loss: 0.4921 | Acc: 0.7625
  [1905/3818] Loss: 0.4928 | Acc: 0.7605
  [1905/3818] Loss: 0.4928 | Acc: 0.7605
  [2286/3818] Loss: 0.4913 | Acc: 0.7615
  [2286/3818] Loss: 0.4913 | Acc: 0.7615
  [2667/3818] Loss: 0.4907 | Acc: 0.7610
  [2667/3818] Loss: 0.4907 | Acc: 0.7610
  [3048/3818] Loss: 0.4904 | Acc: 0.7614
  [3048/3818] Loss: 0.4904 | Acc: 0.7614
  [3429/3818] Loss: 0.4891 | Acc: 0.7622
  [3429/3818] Loss: 0.4891 | Acc: 0.7622
  [3810/3818] Loss: 0.4885 | Acc: 0.7625
  [3810/3818] Loss: 0.4885 | Acc: 0.7625
Train Loss: 0.4885 | Train Acc: 0.7626
Val   Loss: 0.3678 | Val   Acc: 0.8741
Train Loss: 0.4885 | Train Acc: 0.7626
Val   Loss: 0.3678 | Val   Acc: 0.8741
 Saved best model (val_acc: 0.8741)

Epoch 7/10
--------------------------------------
 Saved best model (val_acc: 0.8741)

Epoch 7/10
--------------------------------------
  [ 381/3818] Loss: 0.4886 | Acc: 0.7616
  [ 381/3818] Loss: 0.4886 | Acc: 0.7616
  [ 762/3818] Loss: 0.4886 | Acc: 0.7615
  [ 762/3818] Loss: 0.4886 | Acc: 0.7615
```

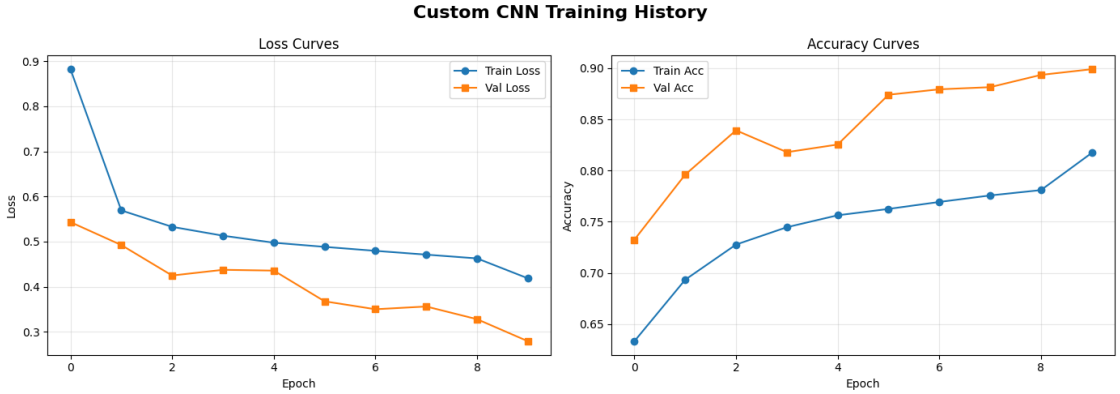
```
  [1143/3818] Loss: 0.4849 | Acc: 0.7646
  [1143/3818] Loss: 0.4849 | Acc: 0.7646
  [1524/3818] Loss: 0.4849 | Acc: 0.7651
  [1524/3818] Loss: 0.4849 | Acc: 0.7651
  [1905/3818] Loss: 0.4846 | Acc: 0.7657
  [1905/3818] Loss: 0.4846 | Acc: 0.7657
  [2286/3818] Loss: 0.4815 | Acc: 0.7681
  [2286/3818] Loss: 0.4815 | Acc: 0.7681
  [2667/3818] Loss: 0.4816 | Acc: 0.7684
  [2667/3818] Loss: 0.4816 | Acc: 0.7684
  [3048/3818] Loss: 0.4809 | Acc: 0.7690
  [3048/3818] Loss: 0.4809 | Acc: 0.7690
  [3429/3818] Loss: 0.4804 | Acc: 0.7692
  [3429/3818] Loss: 0.4804 | Acc: 0.7692
  [3810/3818] Loss: 0.4796 | Acc: 0.7694
  [3810/3818] Loss: 0.4796 | Acc: 0.7694
Train Loss: 0.4796 | Train Acc: 0.7694
Val   Loss: 0.3503 | Val   Acc: 0.8794
Train Loss: 0.4796 | Train Acc: 0.7694
Val   Loss: 0.3503 | Val   Acc: 0.8794
  Saved best model (val_acc: 0.8794)

Epoch 8/10
----------------------------------------
  Saved best model (val_acc: 0.8794)

Epoch 8/10
----------------------------------------
  [ 381/3818] Loss: 0.4700 | Acc: 0.7779
  [ 381/3818] Loss: 0.4700 | Acc: 0.7779
  [ 762/3818] Loss: 0.4690 | Acc: 0.7787
  [ 762/3818] Loss: 0.4690 | Acc: 0.7787
  [1143/3818] Loss: 0.4706 | Acc: 0.7770
  [1143/3818] Loss: 0.4706 | Acc: 0.7770
  [1524/3818] Loss: 0.4716 | Acc: 0.7763
  [1524/3818] Loss: 0.4716 | Acc: 0.7763
  [1905/3818] Loss: 0.4720 | Acc: 0.7759
  [1905/3818] Loss: 0.4720 | Acc: 0.7759
  [2286/3818] Loss: 0.4704 | Acc: 0.7770
  [2286/3818] Loss: 0.4704 | Acc: 0.7770
  [2667/3818] Loss: 0.4709 | Acc: 0.7761
  [2667/3818] Loss: 0.4709 | Acc: 0.7761
  [3048/3818] Loss: 0.4709 | Acc: 0.7763
  [3048/3818] Loss: 0.4709 | Acc: 0.7763
  [3429/3818] Loss: 0.4710 | Acc: 0.7760
  [3429/3818] Loss: 0.4710 | Acc: 0.7760
  [3810/3818] Loss: 0.4712 | Acc: 0.7757
  [3810/3818] Loss: 0.4712 | Acc: 0.7757
```

```
Train Loss: 0.4713 | Train Acc: 0.7757
Val   Loss: 0.3564 | Val   Acc: 0.8815
Train Loss: 0.4713 | Train Acc: 0.7757
Val   Loss: 0.3564 | Val   Acc: 0.8815
  Saved best model (val_acc: 0.8815)

Epoch 9/10
----------------------------------------
  Saved best model (val_acc: 0.8815)

Epoch 9/10
----------------------------------------
  [ 381/3818] Loss: 0.4646 | Acc: 0.7798
  [ 381/3818] Loss: 0.4646 | Acc: 0.7798
  [ 762/3818] Loss: 0.4628 | Acc: 0.7807
  [ 762/3818] Loss: 0.4628 | Acc: 0.7807
  [1143/3818] Loss: 0.4619 | Acc: 0.7816
  [1143/3818] Loss: 0.4619 | Acc: 0.7816
  [1524/3818] Loss: 0.4611 | Acc: 0.7831
  [1524/3818] Loss: 0.4611 | Acc: 0.7831
  [1905/3818] Loss: 0.4620 | Acc: 0.7820
  [1905/3818] Loss: 0.4620 | Acc: 0.7820
  [2286/3818] Loss: 0.4621 | Acc: 0.7819
  [2286/3818] Loss: 0.4621 | Acc: 0.7819
  [2667/3818] Loss: 0.4626 | Acc: 0.7816
  [2667/3818] Loss: 0.4626 | Acc: 0.7816
  [3048/3818] Loss: 0.4634 | Acc: 0.7805
  [3048/3818] Loss: 0.4634 | Acc: 0.7805
  [3429/3818] Loss: 0.4634 | Acc: 0.7808
  [3429/3818] Loss: 0.4634 | Acc: 0.7808
  [3810/3818] Loss: 0.4629 | Acc: 0.7810
  [3810/3818] Loss: 0.4629 | Acc: 0.7810
Train Loss: 0.4629 | Train Acc: 0.7810
Val   Loss: 0.3282 | Val   Acc: 0.8935
Train Loss: 0.4629 | Train Acc: 0.7810
Val   Loss: 0.3282 | Val   Acc: 0.8935
  Saved best model (val_acc: 0.8935)

Epoch 10/10
----------------------------------------
  Saved best model (val_acc: 0.8935)

Epoch 10/10
----------------------------------------
  [ 381/3818] Loss: 0.4641 | Acc: 0.7773
  [ 381/3818] Loss: 0.4641 | Acc: 0.7773
  [ 762/3818] Loss: 0.4597 | Acc: 0.7832
  [ 762/3818] Loss: 0.4597 | Acc: 0.7832
```

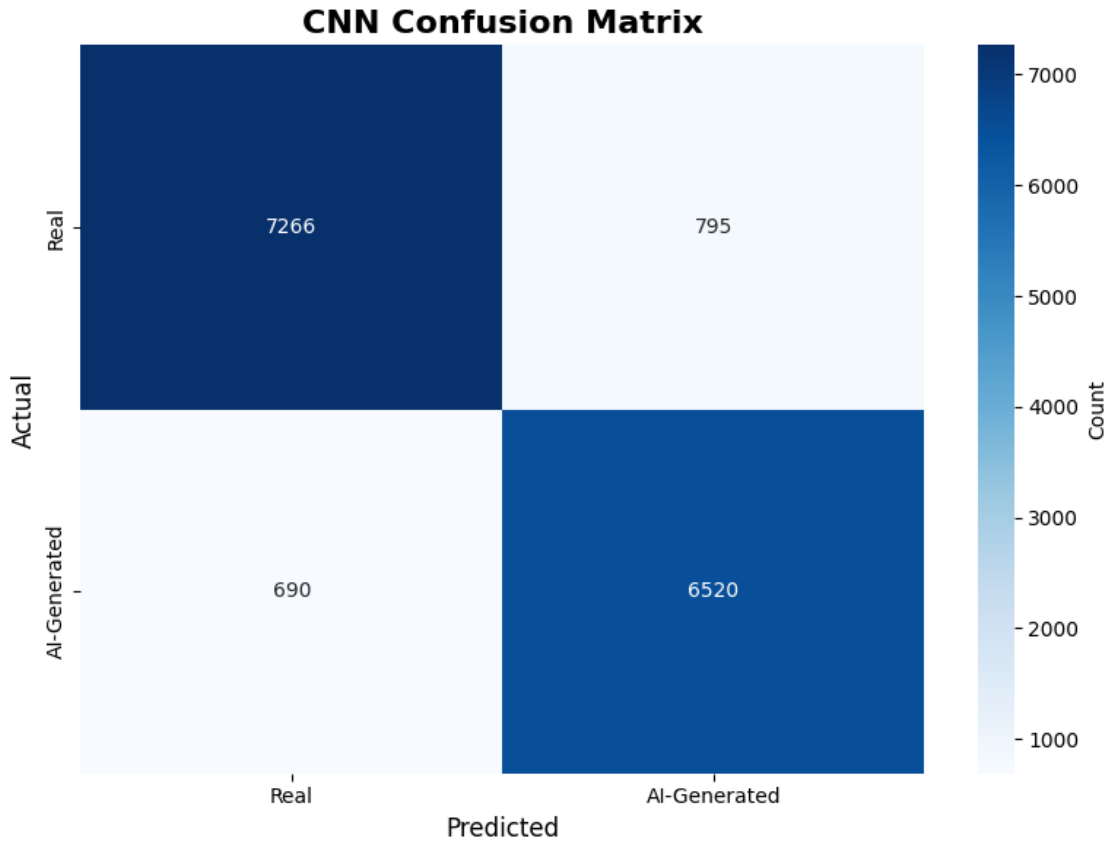
```
  [1143/3818] Loss: 0.4584 | Acc: 0.7846
  [1143/3818] Loss: 0.4584 | Acc: 0.7846
  [1524/3818] Loss: 0.4582 | Acc: 0.7850
  [1524/3818] Loss: 0.4582 | Acc: 0.7850
  [1905/3818] Loss: 0.4532 | Acc: 0.7893
  [1905/3818] Loss: 0.4532 | Acc: 0.7893
  [2286/3818] Loss: 0.4432 | Acc: 0.7979
  [2286/3818] Loss: 0.4432 | Acc: 0.7979
  [2667/3818] Loss: 0.4369 | Acc: 0.8039
  [2667/3818] Loss: 0.4369 | Acc: 0.8039
  [3048/3818] Loss: 0.4314 | Acc: 0.8081
  [3048/3818] Loss: 0.4314 | Acc: 0.8081
  [3429/3818] Loss: 0.4241 | Acc: 0.8133
  [3429/3818] Loss: 0.4241 | Acc: 0.8133
  [3810/3818] Loss: 0.4189 | Acc: 0.8173
  [3810/3818] Loss: 0.4189 | Acc: 0.8173
Train Loss: 0.4187 | Train Acc: 0.8174
Val   Loss: 0.2794 | Val   Acc: 0.8991
Train Loss: 0.4187 | Train Acc: 0.8174
Val   Loss: 0.2794 | Val   Acc: 0.8991
  Saved best model (val_acc: 0.8991)

  Training complete in 5718s
Best validation accuracy: 0.8991
CPU times: user 17min 3s, sys: 3min 20s, total: 20min 23s
Wall time: 1h 35min 18s
  Saved best model (val_acc: 0.8991)

  Training complete in 5718s
Best validation accuracy: 0.8991
CPU times: user 17min 3s, sys: 3min 20s, total: 20min 23s
Wall time: 1h 35min 18s
```

[38]:
```python
# Plot training history
plot_training_history(history_cnn, "Custom CNN Training History")

# Evaluate on test set (convert binary to 2-class for consistency)
# For CNN with BCEWithLogitsLoss, we need special handling
cnn_model.eval()
all_preds_cnn = []
all_labels_cnn = []
with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = cnn_model(images)
        preds = (torch.sigmoid(outputs) > 0.5).long().squeeze()
        all_preds_cnn.extend(preds.cpu().numpy())
```

```
        all_labels_cnn.extend(labels.cpu().numpy())

test_acc_cnn = np.mean(np.array(all_preds_cnn) == np.array(all_labels_cnn))
print(f"\n  Test Accuracy: {test_acc_cnn:.4f}")

plot_confusion_matrix(all_labels_cnn, all_preds_cnn, title='CNN Confusion␣
 ↪Matrix')
print_classification_report(all_labels_cnn, all_preds_cnn)
```




```
Test Accuracy: 0.9028
```




```
============================================================
CLASSIFICATION REPORT
============================================================
              precision    recall  f1-score   support

        Real       0.91      0.90      0.91      8061
AI-Generated       0.89      0.90      0.90      7210

    accuracy                           0.90     15271
   macro avg       0.90      0.90      0.90     15271
weighted avg       0.90      0.90      0.90     15271

              precision    recall  f1-score   support

        Real       0.91      0.90      0.91      8061
AI-Generated       0.89      0.90      0.90      7210

    accuracy                           0.90     15271
   macro avg       0.90      0.90      0.90     15271
weighted avg       0.90      0.90      0.90     15271
```

1.6 5. Model 2: ResNet50 Transfer Learning

Pretrained ResNet50 with frozen backbone and custom classifier.

[39]:
```
%%time
# Create model
resnet_model = get_resnet50(num_classes=2, freeze_backbone=True).to(device)
trainable, total = count_parameters(resnet_model)
print(f"  ResNet50 Parameters: {trainable:,} trainable / {total:,} total")

# Setup training
criterion_resnet = nn.CrossEntropyLoss()
optimizer_resnet = optim.Adam(resnet_model.fc.parameters(), lr=1e-4,␣
 ↪weight_decay=1e-4)

trainer_resnet = Trainer(resnet_model, device, criterion_resnet,␣
 ↪optimizer_resnet, use_amp=True)

# Train
print("\n  Training ResNet50...")
history_resnet = trainer_resnet.fit(train_loader, val_loader, num_epochs=5,
                                    save_path='models/resnet50_best.pth')
```

```
Downloading: "https://download.pytorch.org/models/resnet50-0676ba61.pth" to
/root/.cache/torch/hub/checkpoints/resnet50-0676ba61.pth

100%|      | 97.8M/97.8M [00:00<00:00, 236MB/s]


  ResNet50 Parameters: 4,098 trainable / 23,512,130 total

  Training ResNet50…

Epoch 1/5
----------------------------------------
  [ 381/3818] Loss: 0.6165 | Acc: 0.6611
  [ 381/3818] Loss: 0.6165 | Acc: 0.6611
  [ 762/3818] Loss: 0.5756 | Acc: 0.7025
  [ 762/3818] Loss: 0.5756 | Acc: 0.7025
  [1143/3818] Loss: 0.5528 | Acc: 0.7211
  [1143/3818] Loss: 0.5528 | Acc: 0.7211
  [1524/3818] Loss: 0.5360 | Acc: 0.7337
  [1524/3818] Loss: 0.5360 | Acc: 0.7337
  [1905/3818] Loss: 0.5245 | Acc: 0.7418
  [1905/3818] Loss: 0.5245 | Acc: 0.7418
  [2286/3818] Loss: 0.5160 | Acc: 0.7477
  [2286/3818] Loss: 0.5160 | Acc: 0.7477
```

```
  [2667/3818] Loss: 0.5079 | Acc: 0.7534
  [2667/3818] Loss: 0.5079 | Acc: 0.7534
  [3048/3818] Loss: 0.5016 | Acc: 0.7578
  [3048/3818] Loss: 0.5016 | Acc: 0.7578
  [3429/3818] Loss: 0.4958 | Acc: 0.7621
  [3429/3818] Loss: 0.4958 | Acc: 0.7621
  [3810/3818] Loss: 0.4914 | Acc: 0.7645
  [3810/3818] Loss: 0.4914 | Acc: 0.7645
Train Loss: 0.4913 | Train Acc: 0.7646
Val   Loss: 0.4752 | Val   Acc: 0.7656
 Saved best model (val_acc: 0.7656)

Epoch 2/5
----------------------------------------
Train Loss: 0.4913 | Train Acc: 0.7646
Val   Loss: 0.4752 | Val   Acc: 0.7656
 Saved best model (val_acc: 0.7656)

Epoch 2/5
----------------------------------------
  [ 381/3818] Loss: 0.4470 | Acc: 0.7920
  [ 381/3818] Loss: 0.4470 | Acc: 0.7920
  [ 762/3818] Loss: 0.4422 | Acc: 0.7953
  [ 762/3818] Loss: 0.4422 | Acc: 0.7953
  [1143/3818] Loss: 0.4405 | Acc: 0.7969
  [1143/3818] Loss: 0.4405 | Acc: 0.7969
  [1524/3818] Loss: 0.4403 | Acc: 0.7964
  [1524/3818] Loss: 0.4403 | Acc: 0.7964
  [1905/3818] Loss: 0.4404 | Acc: 0.7968
  [1905/3818] Loss: 0.4404 | Acc: 0.7968
  [2286/3818] Loss: 0.4400 | Acc: 0.7969
  [2286/3818] Loss: 0.4400 | Acc: 0.7969
  [2667/3818] Loss: 0.4380 | Acc: 0.7982
  [2667/3818] Loss: 0.4380 | Acc: 0.7982
  [3048/3818] Loss: 0.4372 | Acc: 0.7984
  [3048/3818] Loss: 0.4372 | Acc: 0.7984
  [3429/3818] Loss: 0.4362 | Acc: 0.7986
  [3429/3818] Loss: 0.4362 | Acc: 0.7986
  [3810/3818] Loss: 0.4356 | Acc: 0.7988
  [3810/3818] Loss: 0.4356 | Acc: 0.7988
Train Loss: 0.4357 | Train Acc: 0.7987
Val   Loss: 0.4536 | Val   Acc: 0.7817
Train Loss: 0.4357 | Train Acc: 0.7987
Val   Loss: 0.4536 | Val   Acc: 0.7817
 Saved best model (val_acc: 0.7817)

Epoch 3/5
----------------------------------------
```

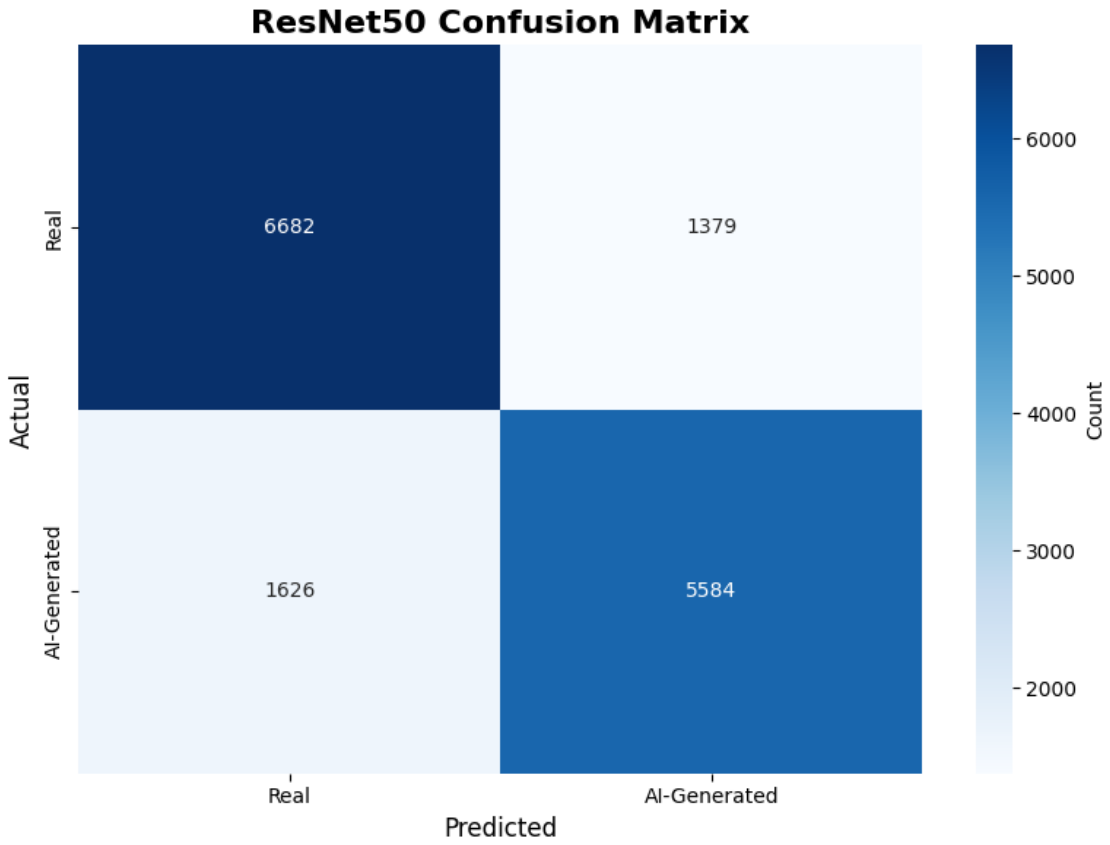
```
  Saved best model (val_acc: 0.7817)

Epoch 3/5
----------------------------------------
  [ 381/3818] Loss: 0.4301 | Acc: 0.8006
  [ 381/3818] Loss: 0.4301 | Acc: 0.8006
  [ 762/3818] Loss: 0.4335 | Acc: 0.7976
  [ 762/3818] Loss: 0.4335 | Acc: 0.7976
  [1143/3818] Loss: 0.4274 | Acc: 0.8007
  [1143/3818] Loss: 0.4274 | Acc: 0.8007
  [1524/3818] Loss: 0.4260 | Acc: 0.8018
  [1524/3818] Loss: 0.4260 | Acc: 0.8018
  [1905/3818] Loss: 0.4249 | Acc: 0.8026
  [1905/3818] Loss: 0.4249 | Acc: 0.8026
  [2286/3818] Loss: 0.4244 | Acc: 0.8031
  [2286/3818] Loss: 0.4244 | Acc: 0.8031
  [2667/3818] Loss: 0.4241 | Acc: 0.8037
  [2667/3818] Loss: 0.4241 | Acc: 0.8037
  [3048/3818] Loss: 0.4237 | Acc: 0.8043
  [3048/3818] Loss: 0.4237 | Acc: 0.8043
  [3429/3818] Loss: 0.4240 | Acc: 0.8041
  [3429/3818] Loss: 0.4240 | Acc: 0.8041
  [3810/3818] Loss: 0.4236 | Acc: 0.8044
  [3810/3818] Loss: 0.4236 | Acc: 0.8044
Train Loss: 0.4236 | Train Acc: 0.8044
Val   Loss: 0.4993 | Val   Acc: 0.7540

Epoch 4/5
----------------------------------------
Train Loss: 0.4236 | Train Acc: 0.8044
Val   Loss: 0.4993 | Val   Acc: 0.7540

Epoch 4/5
----------------------------------------
  [ 381/3818] Loss: 0.4226 | Acc: 0.8083
  [ 381/3818] Loss: 0.4226 | Acc: 0.8083
  [ 762/3818] Loss: 0.4213 | Acc: 0.8095
  [ 762/3818] Loss: 0.4213 | Acc: 0.8095
  [1143/3818] Loss: 0.4193 | Acc: 0.8088
  [1143/3818] Loss: 0.4193 | Acc: 0.8088
  [1524/3818] Loss: 0.4181 | Acc: 0.8091
  [1524/3818] Loss: 0.4181 | Acc: 0.8091
  [1905/3818] Loss: 0.4181 | Acc: 0.8090
  [1905/3818] Loss: 0.4181 | Acc: 0.8090
  [2286/3818] Loss: 0.4174 | Acc: 0.8095
  [2286/3818] Loss: 0.4174 | Acc: 0.8095
  [2667/3818] Loss: 0.4165 | Acc: 0.8101
  [2667/3818] Loss: 0.4165 | Acc: 0.8101
```

```
  [3048/3818] Loss: 0.4154 | Acc: 0.8102
  [3048/3818] Loss: 0.4154 | Acc: 0.8102
  [3429/3818] Loss: 0.4157 | Acc: 0.8101
  [3429/3818] Loss: 0.4157 | Acc: 0.8101
  [3810/3818] Loss: 0.4147 | Acc: 0.8105
  [3810/3818] Loss: 0.4147 | Acc: 0.8105
Train Loss: 0.4147 | Train Acc: 0.8105
Val   Loss: 0.4342 | Val   Acc: 0.7924
Train Loss: 0.4147 | Train Acc: 0.8105
Val   Loss: 0.4342 | Val   Acc: 0.7924
 Saved best model (val_acc: 0.7924)

Epoch 5/5
----------------------------------------
 Saved best model (val_acc: 0.7924)

Epoch 5/5
----------------------------------------
  [ 381/3818] Loss: 0.4066 | Acc: 0.8147
  [ 381/3818] Loss: 0.4066 | Acc: 0.8147
  [ 762/3818] Loss: 0.4081 | Acc: 0.8133
  [ 762/3818] Loss: 0.4081 | Acc: 0.8133
  [1143/3818] Loss: 0.4060 | Acc: 0.8147
  [1143/3818] Loss: 0.4060 | Acc: 0.8147
  [1524/3818] Loss: 0.4057 | Acc: 0.8145
  [1524/3818] Loss: 0.4057 | Acc: 0.8145
  [1905/3818] Loss: 0.4064 | Acc: 0.8145
  [1905/3818] Loss: 0.4064 | Acc: 0.8145
  [2286/3818] Loss: 0.4068 | Acc: 0.8152
  [2286/3818] Loss: 0.4068 | Acc: 0.8152
  [2667/3818] Loss: 0.4073 | Acc: 0.8146
  [2667/3818] Loss: 0.4073 | Acc: 0.8146
  [3048/3818] Loss: 0.4075 | Acc: 0.8147
  [3048/3818] Loss: 0.4075 | Acc: 0.8147
  [3429/3818] Loss: 0.4078 | Acc: 0.8143
  [3429/3818] Loss: 0.4078 | Acc: 0.8143
  [3810/3818] Loss: 0.4075 | Acc: 0.8142
  [3810/3818] Loss: 0.4075 | Acc: 0.8142
Train Loss: 0.4074 | Train Acc: 0.8142
Val   Loss: 0.4166 | Val   Acc: 0.8063
Train Loss: 0.4074 | Train Acc: 0.8142
Val   Loss: 0.4166 | Val   Acc: 0.8063
 Saved best model (val_acc: 0.8063)

 Training complete in 2837s
Best validation accuracy: 0.8063
CPU times: user 6min 28s, sys: 1min 40s, total: 8min 8s
Wall time: 47min 18s
```

```
  Saved best model (val_acc: 0.8063)

  Training complete in 2837s
Best validation accuracy: 0.8063
CPU times: user 6min 28s, sys: 1min 40s, total: 8min 8s
Wall time: 47min 18s
```

[40]:
```python
# Plot and evaluate
plot_training_history(history_resnet, "ResNet50 Training History")

test_acc_resnet, all_preds_resnet, all_labels_resnet = evaluate_model(resnet_model, test_loader, device)
print(f"\n  Test Accuracy: {test_acc_resnet:.4f}")

plot_confusion_matrix(all_labels_resnet, all_preds_resnet, title='ResNet50 Confusion Matrix')
print_classification_report(all_labels_resnet, all_preds_resnet)
```


```
  Test Accuracy: 0.8032
```




```
============================================================
CLASSIFICATION REPORT
============================================================
              precision    recall  f1-score   support

        Real       0.80      0.83      0.82      8061
AI-Generated       0.80      0.77      0.79      7210

    accuracy                           0.80     15271
   macro avg       0.80      0.80      0.80     15271
weighted avg       0.80      0.80      0.80     15271
```

1.7 6. Model 3: EfficientNet-B2 Transfer Learning

Advanced architecture optimized for high-resolution images.

[41]:
```
%%time
# Create model
```

```
efficientnet_model = get_efficientnet_b2(num_classes=2, freeze_backbone=True).
 ↪to(device)
trainable, total = count_parameters(efficientnet_model)
print(f"  EfficientNet-B2 Parameters: {trainable:,} trainable / {total:,}␣
 ↪total")

# Setup training
criterion_eff = nn.CrossEntropyLoss()
optimizer_eff = optim.Adam(efficientnet_model.classifier[1].parameters(),␣
 ↪lr=1e-4, weight_decay=1e-4)

trainer_eff = Trainer(efficientnet_model, device, criterion_eff, optimizer_eff,␣
 ↪use_amp=True)

# Train
print("\n  Training EfficientNet-B2...")
history_eff = trainer_eff.fit(train_loader, val_loader, num_epochs=5,
                              save_path='models/efficientnet_b2_best.pth')
```

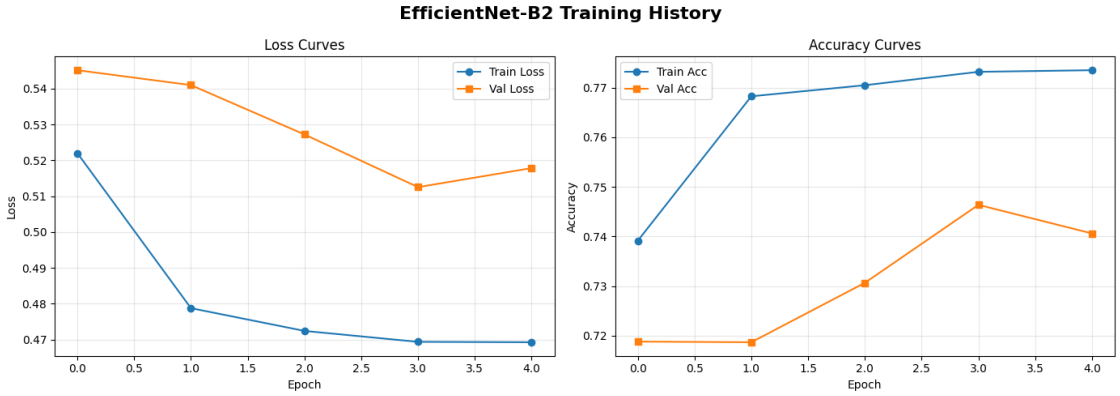
```
Downloading:
"https://download.pytorch.org/models/efficientnet_b2_rwightman-c35c1473.pth" to
/root/.cache/torch/hub/checkpoints/efficientnet_b2_rwightman-c35c1473.pth

100%|      | 35.2M/35.2M [00:00<00:00, 238MB/s]


  EfficientNet-B2 Parameters: 2,818 trainable / 7,703,812 total

  Training EfficientNet-B2…

Epoch 1/5
----------------------------------------
  [ 381/3818] Loss: 0.6346 | Acc: 0.6392
  [ 381/3818] Loss: 0.6346 | Acc: 0.6392
  [ 762/3818] Loss: 0.5988 | Acc: 0.6807
  [ 762/3818] Loss: 0.5988 | Acc: 0.6807
  [1143/3818] Loss: 0.5775 | Acc: 0.6999
  [1143/3818] Loss: 0.5775 | Acc: 0.6999
  [1524/3818] Loss: 0.5625 | Acc: 0.7108
  [1524/3818] Loss: 0.5625 | Acc: 0.7108
  [1905/3818] Loss: 0.5517 | Acc: 0.7177
  [1905/3818] Loss: 0.5517 | Acc: 0.7177
  [2286/3818] Loss: 0.5439 | Acc: 0.7231
  [2286/3818] Loss: 0.5439 | Acc: 0.7231
  [2667/3818] Loss: 0.5374 | Acc: 0.7280
  [2667/3818] Loss: 0.5374 | Acc: 0.7280
  [3048/3818] Loss: 0.5316 | Acc: 0.7328
  [3048/3818] Loss: 0.5316 | Acc: 0.7328
```

```
  [3429/3818] Loss: 0.5263 | Acc: 0.7363
  [3429/3818] Loss: 0.5263 | Acc: 0.7363
  [3810/3818] Loss: 0.5221 | Acc: 0.7391
  [3810/3818] Loss: 0.5221 | Acc: 0.7391
Train Loss: 0.5220 | Train Acc: 0.7391
Val   Loss: 0.5452 | Val   Acc: 0.7188
 Saved best model (val_acc: 0.7188)

Epoch 2/5
----------------------------------------
Train Loss: 0.5220 | Train Acc: 0.7391
Val   Loss: 0.5452 | Val   Acc: 0.7188
 Saved best model (val_acc: 0.7188)

Epoch 2/5
----------------------------------------
  [ 381/3818] Loss: 0.4820 | Acc: 0.7675
  [ 381/3818] Loss: 0.4820 | Acc: 0.7675
  [ 762/3818] Loss: 0.4815 | Acc: 0.7668
  [ 762/3818] Loss: 0.4815 | Acc: 0.7668
  [1143/3818] Loss: 0.4838 | Acc: 0.7651
  [1143/3818] Loss: 0.4838 | Acc: 0.7651
  [1524/3818] Loss: 0.4818 | Acc: 0.7658
  [1524/3818] Loss: 0.4818 | Acc: 0.7658
  [1905/3818] Loss: 0.4815 | Acc: 0.7658
  [1905/3818] Loss: 0.4815 | Acc: 0.7658
  [2286/3818] Loss: 0.4815 | Acc: 0.7666
  [2286/3818] Loss: 0.4815 | Acc: 0.7666
  [2667/3818] Loss: 0.4796 | Acc: 0.7677
  [2667/3818] Loss: 0.4796 | Acc: 0.7677
  [3048/3818] Loss: 0.4792 | Acc: 0.7678
  [3048/3818] Loss: 0.4792 | Acc: 0.7678
  [3429/3818] Loss: 0.4787 | Acc: 0.7683
  [3429/3818] Loss: 0.4787 | Acc: 0.7683
  [3810/3818] Loss: 0.4787 | Acc: 0.7683
  [3810/3818] Loss: 0.4787 | Acc: 0.7683
Train Loss: 0.4788 | Train Acc: 0.7683
Val   Loss: 0.5410 | Val   Acc: 0.7187

Epoch 3/5
----------------------------------------
Train Loss: 0.4788 | Train Acc: 0.7683
Val   Loss: 0.5410 | Val   Acc: 0.7187

Epoch 3/5
----------------------------------------
  [ 381/3818] Loss: 0.4720 | Acc: 0.7737
  [ 381/3818] Loss: 0.4720 | Acc: 0.7737
```

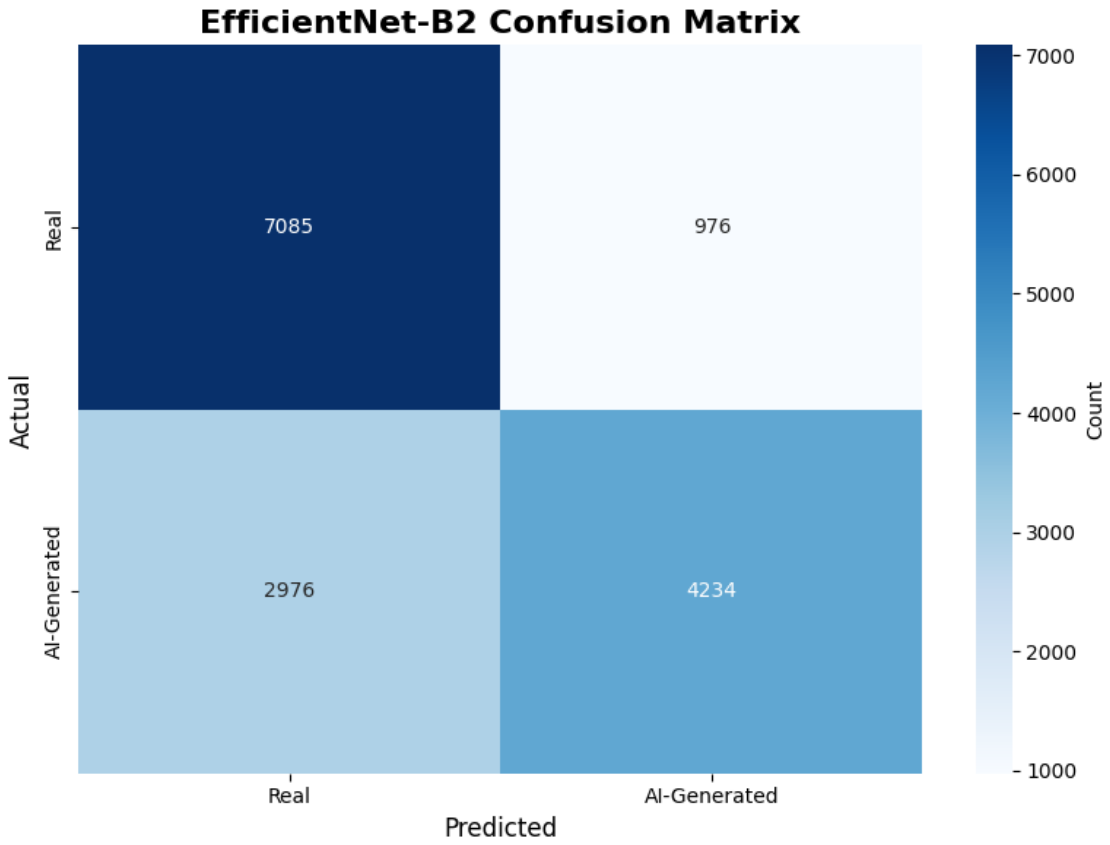
```
  [ 762/3818] Loss: 0.4739 | Acc: 0.7707
  [ 762/3818] Loss: 0.4739 | Acc: 0.7707
  [1143/3818] Loss: 0.4746 | Acc: 0.7697
  [1143/3818] Loss: 0.4746 | Acc: 0.7697
  [1524/3818] Loss: 0.4729 | Acc: 0.7701
  [1524/3818] Loss: 0.4729 | Acc: 0.7701
  [1905/3818] Loss: 0.4727 | Acc: 0.7705
  [1905/3818] Loss: 0.4727 | Acc: 0.7705
  [2286/3818] Loss: 0.4728 | Acc: 0.7705
  [2286/3818] Loss: 0.4728 | Acc: 0.7705
  [2667/3818] Loss: 0.4733 | Acc: 0.7697
  [2667/3818] Loss: 0.4733 | Acc: 0.7697
  [3048/3818] Loss: 0.4731 | Acc: 0.7699
  [3048/3818] Loss: 0.4731 | Acc: 0.7699
  [3429/3818] Loss: 0.4728 | Acc: 0.7703
  [3429/3818] Loss: 0.4728 | Acc: 0.7703
  [3810/3818] Loss: 0.4724 | Acc: 0.7705
  [3810/3818] Loss: 0.4724 | Acc: 0.7705
Train Loss: 0.4724 | Train Acc: 0.7705
Val   Loss: 0.5272 | Val   Acc: 0.7307
  Saved best model (val_acc: 0.7307)

Epoch 4/5
----------------------------------------
Train Loss: 0.4724 | Train Acc: 0.7705
Val   Loss: 0.5272 | Val   Acc: 0.7307
  Saved best model (val_acc: 0.7307)

Epoch 4/5
----------------------------------------
  [ 381/3818] Loss: 0.4699 | Acc: 0.7744
  [ 381/3818] Loss: 0.4699 | Acc: 0.7744
  [ 762/3818] Loss: 0.4697 | Acc: 0.7721
  [ 762/3818] Loss: 0.4697 | Acc: 0.7721
  [1143/3818] Loss: 0.4713 | Acc: 0.7710
  [1143/3818] Loss: 0.4713 | Acc: 0.7710
  [1524/3818] Loss: 0.4696 | Acc: 0.7722
  [1524/3818] Loss: 0.4696 | Acc: 0.7722
  [1905/3818] Loss: 0.4700 | Acc: 0.7724
  [1905/3818] Loss: 0.4700 | Acc: 0.7724
  [2286/3818] Loss: 0.4701 | Acc: 0.7723
  [2286/3818] Loss: 0.4701 | Acc: 0.7723
  [2667/3818] Loss: 0.4699 | Acc: 0.7731
  [2667/3818] Loss: 0.4699 | Acc: 0.7731
  [3048/3818] Loss: 0.4702 | Acc: 0.7729
  [3048/3818] Loss: 0.4702 | Acc: 0.7729
  [3429/3818] Loss: 0.4700 | Acc: 0.7729
  [3429/3818] Loss: 0.4700 | Acc: 0.7729
```

```
  [3810/3818] Loss: 0.4693 | Acc: 0.7733
  [3810/3818] Loss: 0.4693 | Acc: 0.7733
Train Loss: 0.4694 | Train Acc: 0.7732
Val   Loss: 0.5125 | Val   Acc: 0.7464
 Saved best model (val_acc: 0.7464)

Epoch 5/5
----------------------------------------
Train Loss: 0.4694 | Train Acc: 0.7732
Val   Loss: 0.5125 | Val   Acc: 0.7464
 Saved best model (val_acc: 0.7464)

Epoch 5/5
----------------------------------------
  [ 381/3818] Loss: 0.4681 | Acc: 0.7702
  [ 381/3818] Loss: 0.4681 | Acc: 0.7702
  [ 762/3818] Loss: 0.4676 | Acc: 0.7734
  [ 762/3818] Loss: 0.4676 | Acc: 0.7734
  [1143/3818] Loss: 0.4674 | Acc: 0.7742
  [1143/3818] Loss: 0.4674 | Acc: 0.7742
  [1524/3818] Loss: 0.4696 | Acc: 0.7723
  [1524/3818] Loss: 0.4696 | Acc: 0.7723
  [1905/3818] Loss: 0.4682 | Acc: 0.7741
  [1905/3818] Loss: 0.4682 | Acc: 0.7741
  [2286/3818] Loss: 0.4683 | Acc: 0.7736
  [2286/3818] Loss: 0.4683 | Acc: 0.7736
  [2667/3818] Loss: 0.4691 | Acc: 0.7734
  [2667/3818] Loss: 0.4691 | Acc: 0.7734
  [3048/3818] Loss: 0.4689 | Acc: 0.7737
  [3048/3818] Loss: 0.4689 | Acc: 0.7737
  [3429/3818] Loss: 0.4699 | Acc: 0.7729
  [3429/3818] Loss: 0.4699 | Acc: 0.7729
  [3810/3818] Loss: 0.4693 | Acc: 0.7736
  [3810/3818] Loss: 0.4693 | Acc: 0.7736
Train Loss: 0.4693 | Train Acc: 0.7735
Val   Loss: 0.5178 | Val   Acc: 0.7406

 Training complete in 2821s
Best validation accuracy: 0.7464
CPU times: user 10min 11s, sys: 1min 42s, total: 11min 53s
Wall time: 47min 1s
Train Loss: 0.4693 | Train Acc: 0.7735
Val   Loss: 0.5178 | Val   Acc: 0.7406

 Training complete in 2821s
Best validation accuracy: 0.7464
CPU times: user 10min 11s, sys: 1min 42s, total: 11min 53s
Wall time: 47min 1s
```

[42]:
```
# Plot and evaluate
plot_training_history(history_eff, "EfficientNet-B2 Training History")

test_acc_eff, all_preds_eff, all_labels_eff = ␣
 ↪evaluate_model(efficientnet_model, test_loader, device)
print(f"\n Test Accuracy: {test_acc_eff:.4f}")

plot_confusion_matrix(all_labels_eff, all_preds_eff, title='EfficientNet-B2␣
 ↪Confusion Matrix')
print_classification_report(all_labels_eff, all_preds_eff)
```




```
Test Accuracy: 0.7412
```

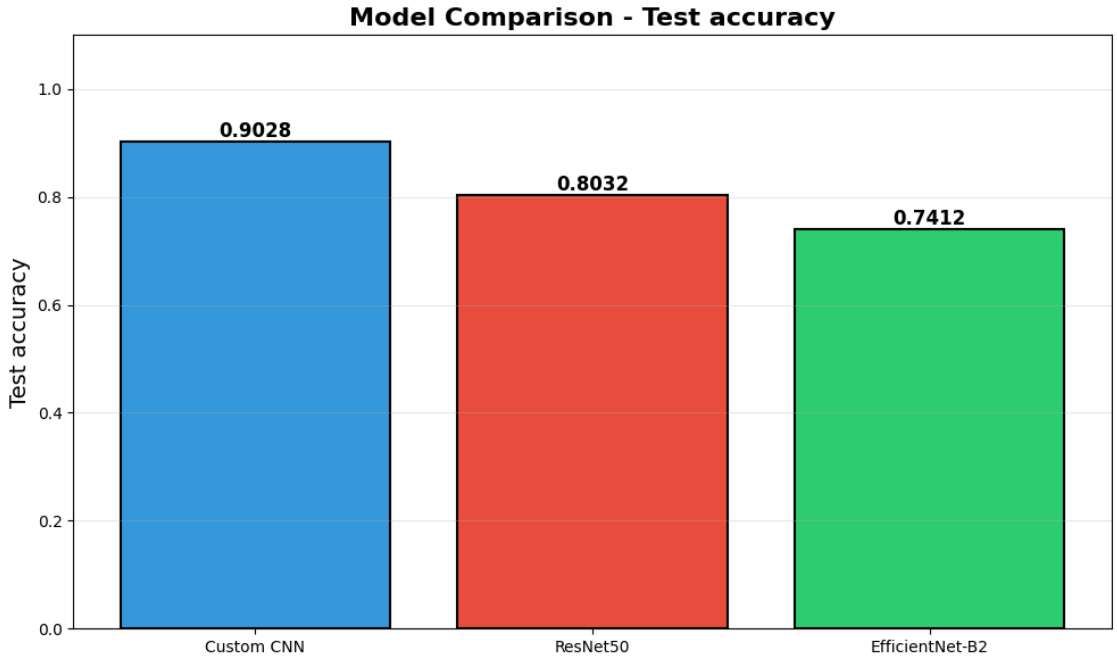



```
============================================================
CLASSIFICATION REPORT
============================================================
              precision    recall  f1-score   support

        Real       0.70      0.88      0.78      8061
AI-Generated       0.81      0.59      0.68      7210

    accuracy                           0.74     15271
   macro avg       0.76      0.73      0.73     15271
weighted avg       0.76      0.74      0.73     15271

              precision    recall  f1-score   support

        Real       0.70      0.88      0.78      8061
AI-Generated       0.81      0.59      0.68      7210

    accuracy                           0.74     15271
   macro avg       0.76      0.73      0.73     15271
weighted avg       0.76      0.74      0.73     15271
```

1.8 7. Model Comparison and Results

Compare all three models side-by-side.

[43]:
```
# Compare test accuracies
results = {
    'Custom CNN': test_acc_cnn,
    'ResNet50': test_acc_resnet,
    'EfficientNet-B2': test_acc_eff
}

compare_models(results, metric='Test Accuracy')

print("\n" + "="*60)
print("FINAL RESULTS SUMMARY")
print("="*60)
for model_name, accuracy in results.items():
    print(f"{model_name:20s}: {accuracy:.4f} ({accuracy*100:.2f}%)")
print("="*60)

# Identify best model
best_model_name = max(results, key=results.get)
best_accuracy = results[best_model_name]
print(f"\n Best Model: {best_model_name} with {best_accuracy:.4f} accuracy")
```



```
============================================================
FINAL RESULTS SUMMARY
============================================================
Custom CNN          : 0.9028 (90.28%)
ResNet50            : 0.8032 (80.32%)
EfficientNet-B2     : 0.7412 (74.12%)
============================================================

  Best Model: Custom CNN with 0.9028 accuracy
```

1.9 8. Conclusion and Next Steps

1.9.1 Key Findings

1. **Custom CNN**: Baseline performance with lightweight architecture
2. **ResNet50**: Strong transfer learning performance with minimal training
3. **EfficientNet-B2**: Best for capturing fine-grained details in high-resolution images

1.9.2 Model Selection Criteria

- **Speed**: Custom CNN (smallest, fastest inference)
- **Accuracy**: EfficientNet-B2 (typically best for this task)
- **Balance**: ResNet50 (good accuracy, reasonable speed)

1.9.3 Future Improvements

1. **Data Augmentation**: Add more sophisticated augmentation techniques
2. **Fine-tuning**: Unfreeze backbone layers for transfer learning models
3. **Ensemble**: Combine predictions from multiple models
4. **Attention Mechanisms**: Add attention layers to focus on synthetic artifacts
5. **Larger Dataset**: Train on more diverse AI-generated images

1.9.4 Deployment Recommendations

- **Production**: Use EfficientNet-B2 for best accuracy
- **Edge Devices**: Use Custom CNN for resource-constrained environments
- **Cloud API**: Use ResNet50 for balanced performance

Appendix B

Source Code

This appendix contains the complete implementation code for the project, organized into five main modules. Model, EDA, training and evaluation modules were abstracted out for clarity. They are imported into the main notebook for use and are included in the appendix for reference.

Data Loading Module (data.py)

```
"""
Data loading and preprocessing utilities.

Contains:
- HuggingFaceImageDataset: Custom PyTorch Dataset
- get_transforms: Create train/val data transforms
- prepare_data: Load and split dataset
"""

import random
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
from datasets import load_dataset


class HuggingFaceImageDataset(Dataset):
    """
    Custom PyTorch Dataset wrapper for Hugging Face images.

    Args:
        images (list): List of PIL images
        labels (list): List of corresponding labels (0=Real, 1=AI-
            Generated)
        transform (callable, optional): Optional transform to apply to
            images
    """
    def __init__(self, images, labels, transform=None):
        self.images = images
        self.labels = labels
        self.transform = transform

    def __len__(self):
        return len(self.images)

    def __getitem__(self, idx):
        image = self.images[idx]
        label = self.labels[idx]

        # Handle palette images with transparency
        if image.mode == 'P' and 'transparency' in image.info:
            image = image.convert('RGBA')
```

```
40
41          if self.transform:
42              image = self.transform(image)
43
44          return image, label
45
46
47  def get_transforms(img_size=224, augment=True):
48      """
49      Get image transformation pipelines.
50
51      Args:
52          img_size (int): Target image size (default: 224)
53          augment (bool): Apply data augmentation (for training)
54
55      Returns:
56          torchvision.transforms.Compose: Transform pipeline
57      """
58      # ImageNet normalization constants
59      mean = [0.485, 0.456, 0.406]
60      std = [0.229, 0.224, 0.225]
61
62      if augment:
63          # Training transforms with augmentation
64          transform = transforms.Compose([
65              transforms.Resize((img_size, img_size)),
66              transforms.Lambda(lambda img: img.convert("RGB")),
67              transforms.RandomHorizontalFlip(p=0.5),
68              transforms.RandomRotation(10),
69              transforms.ColorJitter(
70                  brightness=0.1,
71                  contrast=0.1,
72                  saturation=0.1,
73                  hue=0.05
74              ),
75              transforms.ToTensor(),
76              transforms.Normalize(mean=mean, std=std)
77          ])
78      else:
79          # Validation/test transforms (no augmentation)
80          transform = transforms.Compose([
81              transforms.Resize((img_size, img_size)),
82              transforms.Lambda(lambda img: img.convert("RGB")),
83              transforms.ToTensor(),
84              transforms.Normalize(mean=mean, std=std)
85          ])
86
87      return transform
88
89
90  def prepare_data(dataset_name="Hemg/AI-Generated-vs-Real-Images-
        Datasets",
91                   train_ratio=0.8,
92                   val_ratio=0.1,
```

```
                    seed=42,
                    cache_dir="/root/.cache/huggingface/datasets"):
    """
    Load and split Hugging Face dataset.

    Args:
        dataset_name (str): Hugging Face dataset identifier
        train_ratio (float): Proportion for training (default: 0.8)
        val_ratio (float): Proportion for validation (default: 0.1)
        seed (int): Random seed for reproducibility (default: 42)
        cache_dir (str): Directory to cache downloaded dataset

    Returns:
        tuple: (train_images, train_labels, val_images, val_labels,
                test_images, test_labels)
    """
    import os

    # Check if dataset is already cached
    cache_exists = os.path.exists(cache_dir) and any(
        dataset_name.replace('/', '___') in d
        for d in os.listdir(cache_dir)
        if os.path.isdir(os.path.join(cache_dir, d)))

    if cache_exists:
        print(f"Loading cached dataset '{dataset_name}'...")
    else:
        print(f"Downloading dataset '{dataset_name}'...")
        print("First download: ~2-3 min, then cached for future runs")

    dataset = load_dataset(dataset_name, token=False, cache_dir=
       cache_dir)
    print(f"Dataset loaded: {len(dataset['train']):,} total images")

    # Extract images and labels
    print("Preparing data splits...")
    images = [x['image'] for x in dataset['train']]
    labels = [x['label'] for x in dataset['train']]

    # Shuffle with seed
    random.seed(seed)
    combined = list(zip(images, labels))
    random.shuffle(combined)
    images, labels = zip(*combined)

    # Calculate split indices
    total_size = len(images)
    train_end = int(train_ratio * total_size)
    val_end = int((train_ratio + val_ratio) * total_size)

    # Split data
    train_images = images[:train_end]
    val_images = images[train_end:val_end]
    test_images = images[val_end:]
```

```
146
147     train_labels = labels[:train_end]
148     val_labels = labels[train_end:val_end]
149     test_labels = labels[val_end:]
150
151     return (train_images, train_labels,
152             val_images, val_labels,
153             test_images, test_labels)
154
155
156 def create_dataloaders(train_images, train_labels,
157                        val_images, val_labels,
158                        test_images, test_labels,
159                        batch_size=32,
160                        num_workers=2):
161     """
162     Create PyTorch DataLoaders from image/label lists.
163
164     Args:
165         train_images, train_labels: Training data
166         val_images, val_labels: Validation data
167         test_images, test_labels: Test data
168         batch_size (int): Batch size for dataloaders (default: 32)
169         num_workers (int): Number of worker processes (default: 2)
170
171     Returns:
172         tuple: (train_loader, val_loader, test_loader)
173     """
174     # Get transforms
175     train_transform = get_transforms(augment=True)
176     val_transform = get_transforms(augment=False)
177
178     # Create datasets
179     train_dataset = HuggingFaceImageDataset(
180         train_images, train_labels, train_transform)
181     val_dataset = HuggingFaceImageDataset(
182         val_images, val_labels, val_transform)
183     test_dataset = HuggingFaceImageDataset(
184         test_images, test_labels, val_transform)
185
186     # Create dataloaders
187     train_loader = DataLoader(
188         train_dataset,
189         batch_size=batch_size,
190         shuffle=True,
191         num_workers=num_workers,
192         pin_memory=True
193     )
194
195     val_loader = DataLoader(
196         val_dataset,
197         batch_size=batch_size,
198         shuffle=False,
199         num_workers=num_workers,
```

```
        pin_memory=True
    )

    test_loader = DataLoader(
        test_dataset,
        batch_size=batch_size,
        shuffle=False,
        num_workers=num_workers,
        pin_memory=True
    )

    return train_loader, val_loader, test_loader
```

Model Architectures (models.py)

```
"""
Model architectures for AI-generated image detection.

Contains:
- CustomCNN: Baseline CNN with 3 conv layers
- get_resnet50: ResNet50 with transfer learning
- get_efficientnet_b2: EfficientNet-B2 with transfer learning
"""

import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision import models


class CustomCNN(nn.Module):
    """
    Custom CNN architecture for binary image classification.

    Architecture:
        - 3 convolutional layers (32, 64, 128 filters)
        - Batch normalization after each conv layer
        - MaxPooling after each conv block
        - Dropout for regularization
        - 2 fully connected layers

    Args:
        num_classes (int): Number of output classes (default: 1 for
            binary)
    """
    def __init__(self, num_classes=1):
        super(CustomCNN, self).__init__()

        # Convolutional layers
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm2d(32)

```

```
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(64)

        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
        self.bn3 = nn.BatchNorm2d(128)

        # Pooling and dropout
        self.pool = nn.MaxPool2d(2, 2)
        self.dropout = nn.Dropout(0.25)

        # Fully connected layers
        # After 3 pooling layers: 224/8 = 28, so 128 * 28 * 28
        self.fc1 = nn.Linear(128 * 28 * 28, 512)
        self.fc2 = nn.Linear(512, num_classes)

    def forward(self, x):
        # Conv block 1
        x = self.pool(F.relu(self.bn1(self.conv1(x))))

        # Conv block 2
        x = self.pool(F.relu(self.bn2(self.conv2(x))))

        # Conv block 3
        x = self.pool(F.relu(self.bn3(self.conv3(x))))

        # Flatten
        x = x.view(-1, 128 * 28 * 28)

        # Fully connected layers
        x = self.dropout(F.relu(self.fc1(x)))
        x = self.fc2(x)

        return x


def get_resnet50(num_classes=2, pretrained=True, freeze_backbone=True):
    """
    Get ResNet50 model with custom classifier for transfer learning.

    Args:
        num_classes (int): Number of output classes (default: 2)
        pretrained (bool): Use ImageNet pretrained weights (default:
            True)
        freeze_backbone (bool): Freeze all layers except classifier

    Returns:
        torch.nn.Module: ResNet50 model
    """
    # Load pretrained ResNet50
    if pretrained:
        model = models.resnet50(
            weights=models.ResNet50_Weights.IMAGENET1K_V1)
    else:
        model = models.resnet50(weights=None)
```

```
90
91      # Freeze backbone if requested
92      if freeze_backbone:
93          for param in model.parameters():
94              param.requires_grad = False
95
96      # Replace final fully connected layer
97      num_features = model.fc.in_features
98      model.fc = nn.Linear(num_features, num_classes)
99
100     return model
101
102
103 def get_efficientnet_b2(num_classes=2, pretrained=True,
104                         freeze_backbone=True):
105     """
106     Get EfficientNet-B2 model with custom classifier for transfer
            learning.
107
108     Args:
109         num_classes (int): Number of output classes (default: 2)
110         pretrained (bool): Use ImageNet pretrained weights (default:
                True)
111         freeze_backbone (bool): Freeze all layers except classifier
112
113     Returns:
114         torch.nn.Module: EfficientNet-B2 model
115     """
116     # Load pretrained EfficientNet-B2
117     if pretrained:
118         model = models.efficientnet_b2(
119             weights=models.EfficientNet_B2_Weights.IMAGENET1K_V1)
120     else:
121         model = models.efficientnet_b2(weights=None)
122
123     # Freeze backbone if requested
124     if freeze_backbone:
125         for param in model.parameters():
126             param.requires_grad = False
127
128     # Replace classifier head
129     num_features = model.classifier[1].in_features
130     model.classifier[1] = nn.Linear(num_features, num_classes)
131
132     return model
133
134
135 def count_parameters(model):
136     """
137     Count trainable and total parameters in a model.
138
139     Args:
140         model (torch.nn.Module): PyTorch model
141
```

```
    Returns:
        tuple: (trainable_params, total_params)
    """
    trainable = sum(p.numel() for p in model.parameters()
                    if p.requires_grad)
    total = sum(p.numel() for p in model.parameters())
    return trainable, total
```

Training Module (training.py)

```
"""
Training and evaluation utilities.

Contains:
- Trainer: Main training class with AMP support
- evaluate_model: Model evaluation function
- save_checkpoint/load_checkpoint: Model persistence
"""

import os
import copy
import time
import torch
import torch.nn as nn
import torch.optim as optim
from pathlib import Path


class Trainer:
    """
    Training manager with automatic mixed precision (AMP) support.

    Args:
        model (torch.nn.Module): Model to train
        device (torch.device): Device to train on
        criterion (torch.nn.Module): Loss function
        optimizer (torch.optim.Optimizer): Optimizer
        use_amp (bool): Use automatic mixed precision (default: True)
    """
    def __init__(self, model, device, criterion, optimizer, use_amp=
        True):
        self.model = model
        self.device = device
        self.criterion = criterion
        self.optimizer = optimizer
        self.use_amp = use_amp

        # Initialize AMP scaler if using CUDA
        if use_amp and device.type == 'cuda':
            self.scaler = torch.amp.GradScaler('cuda')
        else:
            self.scaler = None
```

```
42
43          # Training history
44          self.history = {
45              'train_loss': [],
46              'train_acc': [],
47              'val_loss': [],
48              'val_acc': []
49          }
50
51          self.best_val_acc = 0.0
52          self.best_model_wts = None
53
54      def train_epoch(self, dataloader, verbose=True):
55          """Train for one epoch."""
56          self.model.train()
57          running_loss = 0.0
58          running_corrects = 0
59          total = 0
60
61          num_batches = len(dataloader)
62          print_every = max(num_batches // 10, 100)
63
64          for batch_idx, (images, labels) in enumerate(dataloader):
65              images = images.to(self.device, non_blocking=True)
66              labels = labels.to(self.device, non_blocking=True)
67
68              self.optimizer.zero_grad()
69
70              # Forward pass with AMP
71              if self.scaler is not None:
72                  with torch.amp.autocast('cuda'):
73                      outputs = self.model(images)
74                      if outputs.dim() == 2 and outputs.size(1) == 1:
75                          labels_for_loss = labels.float().unsqueeze(1)
76                      else:
77                          labels_for_loss = labels
78                      loss = self.criterion(outputs, labels_for_loss)
79
80                  # Backward pass with scaler
81                  self.scaler.scale(loss).backward()
82                  self.scaler.step(self.optimizer)
83                  self.scaler.update()
84              else:
85                  outputs = self.model(images)
86                  if outputs.dim() == 2 and outputs.size(1) == 1:
87                      labels_for_loss = labels.float().unsqueeze(1)
88                  else:
89                      labels_for_loss = labels
90                  loss = self.criterion(outputs, labels_for_loss)
91                  loss.backward()
92                  self.optimizer.step()
93
94              # Calculate accuracy
95              batch_size = labels.size(0)
```

```
96
97              if outputs.dim() == 1 or outputs.size(1) == 1:
98                  preds = (torch.sigmoid(outputs.squeeze()) > 0.5).long()
99              else:
100                 _, preds = torch.max(outputs, 1)
101
102             running_loss += loss.item() * batch_size
103             running_corrects += torch.sum(preds == labels).item()
104             total += batch_size
105
106             # Print progress
107             if verbose and (batch_idx + 1) % print_every == 0:
108                 batch_acc = running_corrects / total
109                 avg_loss = running_loss / total
110                 print(f"  [{batch_idx+1:4d}/{num_batches}] "
111                       f"Loss: {avg_loss:.4f} | Acc: {batch_acc:.4f}")
112
113         epoch_loss = running_loss / total
114         epoch_acc = running_corrects / total
115
116         return epoch_loss, epoch_acc
117
118     def validate(self, dataloader):
119         """Validate model on validation set."""
120         self.model.eval()
121         running_loss = 0.0
122         running_corrects = 0
123         total = 0
124
125         with torch.no_grad():
126             for images, labels in dataloader:
127                 images = images.to(self.device, non_blocking=True)
128                 labels = labels.to(self.device, non_blocking=True)
129
130                 outputs = self.model(images)
131                 if outputs.dim() == 2 and outputs.size(1) == 1:
132                     labels_for_loss = labels.float().unsqueeze(1)
133                 else:
134                     labels_for_loss = labels
135                 loss = self.criterion(outputs, labels_for_loss)
136
137                 batch_size = labels.size(0)
138
139                 if outputs.dim() == 1 or outputs.size(1) == 1:
140                     preds = (torch.sigmoid(outputs.squeeze()) > 0.5).
                            long()
141                 else:
142                     _, preds = torch.max(outputs, 1)
143
144                 running_loss += loss.item() * batch_size
145                 running_corrects += torch.sum(preds == labels).item()
146                 total += batch_size
147
148         epoch_loss = running_loss / total
```

```
        epoch_acc = running_corrects / total

        return epoch_loss, epoch_acc

    def fit(self, train_loader, val_loader, num_epochs,
            save_path=None, verbose=True):
        """Train model for multiple epochs."""
        start_time = time.time()

        for epoch in range(num_epochs):
            if verbose:
                print(f"\\nEpoch {epoch+1}/{num_epochs}")
                print("-" * 40)

            # Train
            train_loss, train_acc = self.train_epoch(
                train_loader, verbose=verbose)
            self.history['train_loss'].append(train_loss)
            self.history['train_acc'].append(train_acc)

            # Validate
            val_loss, val_acc = self.validate(val_loader)
            self.history['val_loss'].append(val_loss)
            self.history['val_acc'].append(val_acc)

            if verbose:
                print(f"Train Loss: {train_loss:.4f} | "
                      f"Train Acc: {train_acc:.4f}")
                print(f"Val   Loss: {val_loss:.4f} | "
                      f"Val   Acc: {val_acc:.4f}")

            # Save best model
            if val_acc > self.best_val_acc:
                self.best_val_acc = val_acc
                self.best_model_wts = copy.deepcopy(
                    self.model.state_dict())

                if save_path:
                    save_checkpoint(self.model, save_path)
                    if verbose:
                        print(f"Saved best model "
                              f"(val_acc: {val_acc:.4f})")

        # Load best weights
        if self.best_model_wts is not None:
            self.model.load_state_dict(self.best_model_wts)

        elapsed = time.time() - start_time
        if verbose:
            print(f"\\nTraining complete in {elapsed:.0f}s")
            print(f"Best validation accuracy: {self.best_val_acc:.4f}")

        return self.history

```

```python

def evaluate_model(model, dataloader, device):
    """Evaluate model on a dataset."""
    model.eval()
    all_preds = []
    all_labels = []
    correct = 0
    total = 0

    with torch.no_grad():
        for images, labels in dataloader:
            images = images.to(device)
            labels = labels.to(device)

            outputs = model(images)
            _, preds = torch.max(outputs, 1)

            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())

            correct += (preds == labels).sum().item()
            total += labels.size(0)

    accuracy = correct / total
    return accuracy, all_preds, all_labels


def save_checkpoint(model, path):
    """Save model checkpoint."""
    Path(path).parent.mkdir(parents=True, exist_ok=True)
    torch.save(model.state_dict(), path)


def load_checkpoint(model, path, device):
    """Load model checkpoint."""
    model.load_state_dict(torch.load(path, map_location=device))
    return model
```

Visualization Module (visualization.py)

```python
"""
Visualization utilities for model evaluation and analysis.

Contains:
- plot_training_history: Plot loss and accuracy curves
- plot_confusion_matrix: Display confusion matrix
- compare_models: Side-by-side model comparison
"""

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.metrics import confusion_matrix, classification_report


def plot_training_history(history, title="Training History"):
    """Plot training and validation loss/accuracy curves."""
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))

    # Loss plot
    axes[0].plot(history['train_loss'], label='Train Loss', marker='o')
    axes[0].plot(history['val_loss'], label='Val Loss', marker='s')
    axes[0].set_xlabel('Epoch')
    axes[0].set_ylabel('Loss')
    axes[0].set_title('Loss Curves')
    axes[0].legend()
    axes[0].grid(alpha=0.3)

    # Accuracy plot
    axes[1].plot(history['train_acc'], label='Train Acc', marker='o')
    axes[1].plot(history['val_acc'], label='Val Acc', marker='s')
    axes[1].set_xlabel('Epoch')
    axes[1].set_ylabel('Accuracy')
    axes[1].set_title('Accuracy Curves')
    axes[1].legend()
    axes[1].grid(alpha=0.3)

    plt.suptitle(title, fontsize=16, fontweight='bold')
    plt.tight_layout()
    plt.show()


def plot_confusion_matrix(y_true, y_pred,
                          class_names=['Real', 'AI-Generated'],
                          title='Confusion Matrix'):
    """Plot confusion matrix heatmap."""
    cm = confusion_matrix(y_true, y_pred)

    plt.figure(figsize=(8, 6))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=class_names,
                yticklabels=class_names,
                cbar_kws={'label': 'Count'})
    plt.title(title, fontsize=16, fontweight='bold')
    plt.xlabel('Predicted', fontsize=12)
    plt.ylabel('Actual', fontsize=12)
    plt.tight_layout()
    plt.show()


def compare_models(results_dict, metric='accuracy'):
    """Create bar chart comparing multiple models."""
    models = list(results_dict.keys())
    values = list(results_dict.values())

    plt.figure(figsize=(10, 6))
```

```
    colors = ['#3498db', '#e74c3c', '#2ecc71']
    bars = plt.bar(models, values, color=colors[:len(models)],
                   edgecolor='black', linewidth=1.5)

    # Add value labels on bars
    for bar in bars:
        height = bar.get_height()
        plt.text(bar.get_x() + bar.get_width()/2., height,
                f'{height:.4f}',
                ha='center', va='bottom', fontsize=12,
                fontweight='bold')

    plt.ylabel(metric.capitalize(), fontsize=14)
    plt.title(f'Model Comparison - {metric.capitalize()}',
              fontsize=16, fontweight='bold')
    plt.ylim(0, 1.1)
    plt.grid(axis='y', alpha=0.3)
    plt.tight_layout()
    plt.show()
```

Exploratory Data Analysis Module (eda.py)

```
"""
Exploratory Data Analysis (EDA) utilities.

Contains functions for visualizing and analyzing
the dataset before training.
"""

import random
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter


def analyze_class_distribution(labels,
                               class_names=['Real', 'AI-Generated']):
    """Analyze and print class distribution statistics."""
    unique, counts = np.unique(labels, return_counts=True)
    total = len(labels)

    stats = {}
    print("Class Distribution:")
    for label, count in zip(unique, counts):
        class_name = class_names[label] if label < len(class_names) \
                     else f"Class {label}"
        percentage = (count / total) * 100
        stats[class_name] = {'count': count, 'percentage': percentage}
        print(f"   {class_name}: {count:,} images ({percentage:.1f}%)")

    return stats

```

```
31
32 def plot_class_distribution(labels,
33                             class_names=['Real', 'AI-Generated'],
34                             title='Class Distribution',
35                             figsize=(8, 5)):
36     """Plot class distribution as a bar chart."""
37     unique, counts = np.unique(labels, return_counts=True)
38
39     plt.figure(figsize=figsize)
40     colors = ['#3498db', '#e74c3c', '#2ecc71', '#f39c12'][:len(unique)]
41     bars = plt.bar(class_names[:len(unique)], counts, color=colors,
42                    edgecolor='black', linewidth=1.5)
43
44     plt.title(title, fontsize=16, fontweight='bold')
45     plt.ylabel('Number of Images', fontsize=12)
46     plt.xlabel('Class', fontsize=12)
47     plt.grid(axis='y', alpha=0.3)
48
49     # Add count labels on bars
50     for bar in bars:
51         height = bar.get_height()
52         plt.text(bar.get_x() + bar.get_width()/2., height,
53                 f'{int(height):,}',
54                 ha='center', va='bottom', fontsize=12,
55                 fontweight='bold')
56
57     plt.tight_layout()
58     plt.show()
59
60
61 def visualize_samples(images, labels,
62                       class_names=['Real', 'AI-Generated'],
63                       n_per_class=4, figsize=(16, 8), seed=None):
64     """Display sample images from each class in a grid."""
65     if seed is not None:
66         random.seed(seed)
67
68     # Separate images by class
69     classes = {}
70     for img, lbl in zip(images, labels):
71         if lbl not in classes:
72             classes[lbl] = []
73         classes[lbl].append(img)
74
75     # Create subplot grid
76     n_classes = len(classes)
77     fig, axes = plt.subplots(n_classes, n_per_class, figsize=figsize)
78
79     # Plot samples for each class
80     for class_idx, (label, class_images) in enumerate(
81             sorted(classes.items())):
82         class_name = class_names[label] if label < len(class_names) \
83                      else f"Class {label}"
84         color = ['#3498db', '#e74c3c', '#2ecc71', '#f39c12'][label % 4]
```

```
85
86          for img_idx in range(n_per_class):
87              if img_idx < len(class_images):
88                  sample_img = random.choice(class_images)
89                  axes[class_idx, img_idx].imshow(sample_img)
90              else:
91                  axes[class_idx, img_idx].axis('off')
92
93              if img_idx == 0:
94                  axes[class_idx, img_idx].set_ylabel(
95                      class_name, fontsize=12, fontweight='bold',
96                      color=color)
97
98              axes[class_idx, img_idx].axis('off')
99
100     plt.tight_layout()
101     plt.show()
102
103
104 def dataset_summary(train_images, train_labels,
105                     val_images, val_labels,
106                     test_images, test_labels,
107                     class_names=['Real', 'AI-Generated']):
108     """Print comprehensive dataset summary statistics."""
109     total_size = len(train_images) + len(val_images) + len(test_images)
110
111     print("\\n" + "="*60)
112     print("DATASET SUMMARY")
113     print("="*60)
114     print(f"\\nTotal Images: {total_size:,}")
115     print(f"\\nSplit:")
116     print(f"   Training:   {len(train_images):,} images "
117           f"({len(train_images)/total_size*100:.1f}%)")
118     print(f"   Validation: {len(val_images):,} images "
119           f"({len(val_images)/total_size*100:.1f}%)")
120     print(f"   Test:       {len(test_images):,} images "
121           f"({len(test_images)/total_size*100:.1f}%)")
122
123     print("="*60)
```